

Politecnico di Torino

Master's Degree in Mechatronics Engineering

Academic Year 2025/2026

Graduation Session December 2026

Visual Autonomous Navigation for Horticulture

Supervisors:
Name Surname
Name Surname

Candidate:
Emanuele Pruiti

Abstract

Precision agriculture aims to increase crop yield and the efficiency of field operations while reducing waste. At the same time, the growing world population increases the demand for agricultural production, while the shortage of skilled labor and the rising cost of farm management push agriculture towards autonomous robots. Row-based crops are among the most studied applications of these robots.

Navigating between crop rows requires precise positioning, because the robot must stay within the corridor without damaging the plants. Satellite positioning alone does not guarantee this precision, since dense canopies and greenhouse structures degrade the GNSS signal and its accuracy. In addition, RTK corrections and 3D LiDAR sensors are expensive, which limits the large-scale adoption of agricultural robots. Camera-based methods are cheaper, but classical pipelines based on color thresholds are sensitive to illumination changes and weeds. Segmentation networks handle these conditions better, but they need large datasets annotated manually at pixel level, which take a long time to produce, and they must run fast on onboard hardware.

In this thesis, we propose a low-cost solution based on a monocular RGB camera and composed of two stages: segmentation and path extraction. In the first stage, a semantic segmentation network predicts the path between the rows. In the second stage, a path extractor estimates from the predicted mask the centerline of this path, which the robot follows to move along the row. The extractor divides the region of interest into horizontal strips, follows the corridor strip by strip from the bottom of the image upwards, and fits the left and right edges separately with iteratively reweighted least squares, so that short gaps and local errors in the predicted mask have a limited effect on the result. The centerline is the mean of the two edge lines.

The FCN, DeepLabV3, and LR-ASPP deep learning architectures were compared on a manually annotated dataset of lettuce images. LR-ASPP was selected for its speed: its accuracy is about 1% lower than that of the other two networks, but its inference is 7 to 11 times faster, providing lightweight execution on the onboard computer.

To reduce the annotation effort, we investigate whether manual annotation

can be replaced by labels generated automatically by SAM 3 from text prompts, without manual correction. In operating conditions, on the images taken between parallel rows, networks trained on SAM 3 labels reach a path Intersection over Union about 6% lower than the network trained on manual labels. Since the path is the only class used for navigation, the robot uses a binary network trained on the SAM 3 labels of lettuce, tomato, and cucumber crops, so the tomato and cucumber images required no manual annotation.

The images were collected in two experimental campaigns, and a third campaign was dedicated to the tests on the robot. The proposed navigation method was compared with three state-of-the-art solutions. The tests showed that the robot can autonomously navigate in horticultural rows with a monocular RGB camera and a segmentation neural network trained using only automatic labels without manual correction. These results represent a possible robust solution for precise motion inside the crop rows and a lower risk of damaging the crops, paving the way for further horticultural operations.

Acknowledgements

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodo sed egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

Table of Contents

List of Figures VIII

1 Introduction 1
- 1.1 Context and motivation . . . 1
- 1.2 Objectives of the thesis . . . 2
- 1.3 Organization of the work . . . 3

2 Background and Problem Formulation 4
- 2.1 Agricultural autonomous navigation . . . 4
  - 2.1.1 Navigation tasks and requirements . . . 4
  - 2.1.2 Environmental challenges . . . 4
- 2.2 Vision-based local guidance . . . 5
  - 2.2.1 Sensing modalities . . . 5
  - 2.2.2 Image-space reference extraction . . . 5
- 2.3 The segmentation-to-control pipeline . . . 7
  - 2.3.1 Perception stage . . . 7
  - 2.3.2 Geometric extraction stage . . . 7
  - 2.3.3 Control integration . . . 8

3 State of the Art 9
- 3.1 Classical image-processing methods . . . 9
- 3.2 Learning-based segmentation for agricultural navigation . . . 9
  - 3.2.1 Semantic segmentation approaches . . . 9
  - 3.2.2 Training data and generalization . . . 10
- 3.3 Semantic segmentation architectures . . . 10
  - 3.3.1 FCN, DeepLabV3, and mobile architectures . . . 10
  - 3.3.2 Transfer learning and pretraining . . . 10
- 3.4 Dataset preparation and model-assisted annotation . . . 11
  - 3.4.1 Manual annotation challenges . . . 11
  - 3.4.2 SAM and foundation models for segmentation . . . 11
- 3.5 Geometric extraction methods . . . 12

3.5.1 Line fitting and crop-edge detection . . . 12
3.5.2 Column-wise histogram methods . . . 12
3.5.3 Horizontal strips and adaptive multi-ROI methods . . . 12
3.5.4 Triangle Scan Method . . . 13
3.5.5 Bilateral boundary tracking versus centroid reduction . . . 13
3.5.6 Comparison of geometric representations . . . 15
3.6 System integration and experimental validation . . . 16
3.6.1 Perception and control integration . . . 16
3.6.2 Scope of reported evaluations . . . 16
3.7 Positioning of this thesis . . . 16

4 Deep Learning Architectures for Semantic Segmentation 18
4.1 Problem formulation and encoder-decoder principles . . . 19
4.2 Fully Convolutional Networks (FCN) . . . 20
4.2.1 Convolutionalization of classification networks . . . 21
4.2.2 Skip architecture and upsampling . . . 21
4.3 DeepLabV3 and atrous convolutions . . . 22
4.3.1 Atrous convolution formulation . . . 23
4.3.2 Atrous Spatial Pyramid Pooling (ASPP) . . . 23
4.4 MobileNetV3 and the Lite Reduced ASPP (LR-ASPP) head . . . 24
4.4.1 Depthwise separable convolutions . . . 25
4.4.2 Inverted residual blocks with linear bottlenecks . . . 25
4.4.3 Squeeze-and-Excitation (SE) channel attention . . . 26
4.4.4 Efficient non-linearities: hard-swish and hard-sigmoid . . . 27
4.4.5 The Lite Reduced ASPP (LR-ASPP) architecture . . . 27
4.5 Segment Anything Model 3 (SAM 3) for dataset annotation . . . 28
4.5.1 Architecture overview . . . 30
4.5.2 Operational distinction from onboard models . . . 31
4.6 Summary of architectural characteristics . . . 31

5 Dataset Preparation and Comparative Model Training 33
5.1 Image acquisition and class definition . . . 34
5.1.1 Image sources and acquisition conditions . . . 34
5.1.2 Semantic classes and labeling targets . . . 34
5.2 Dataset preparation . . . 35
5.2.1 Lettuce dataset with manual annotation . . . 35
5.2.2 Automated zero-shot annotation via SAM 3 . . . 35
5.2.3 Tomato and cucumber images . . . 36
5.2.4 Dataset partitioning . . . 37
5.3 Architecture comparison: FCN, DeepLabV3, and LR-ASPP . . . 37
5.3.1 Model selection and pretrained weights . . . 38

5.3.2 Training configuration and loss formulation 38
5.3.3 Evaluation metrics and results 40
5.3.4 Selection of LR-ASPP 42
5.4 Manual versus SAM 3 labels 42
5.4.1 Training configurations and evaluation 42
5.4.2 Results on the lettuce test images 43
5.4.3 Extension to the multi-crop dataset and deployed network . 45

6 Geometric Extraction of the Local Navigation Reference 46
6.1 Edge and centerline models 46
6.2 Region of interest and mask cleaning 47
6.3 Horizontal strips and intervals 47
6.4 Bottom-up association and chain selection 48
6.5 Separate fitting of the corridor edges 49
6.6 Validation, confidence, and output 50
6.7 Baseline extractors 51
6.7.1 Center points 51
6.7.2 Column histogram 53
6.7.3 RANSAC edges 53
6.8 Structural differences and limitations 54

7 Robot Platform and Integration 56
7.1 Robot hardware and sensors 56
7.1.1 Robot platform 56
7.1.2 Camera configuration 57
7.1.3 Computing platform 59
7.2 Software architecture 59
7.2.1 ROS integration 59
7.2.2 Perception pipeline implementation 62
7.2.3 Control interface 64
7.3 Deployment configuration 64
7.3.1 Model export to ONNX 64
7.3.2 Real-time inference 65
7.3.3 Timing and latency 65
7.3.4 Software-in-the-Loop pre-validation 66

8 Experimental Validation 68
8.1 Offline evaluation protocol 68
8.1.1 Segmentation metrics 68
8.1.2 Geometric reference accuracy 70
8.2 Field validation 72

8.2.1 Ground-truth reference: laser sensor . . . 73
8.2.2 Field evaluation metrics . . . 73
8.2.3 Test conditions . . . 75
8.2.4 Closed-loop navigation results . . . 75
8.2.5 Qualitative observations . . . 76
8.3 Discussion . . . 76
8.3.1 Segmentation performance . . . 76
8.3.2 Geometric extraction trade-offs . . . 76
8.3.3 Limitations . . . 77

9 Conclusions and Future Work 79
9.1 Main contributions . . . 79
9.2 Conclusions on segmentation and annotation . . . 80
9.3 Conclusions on geometric extraction . . . 80
9.4 Limitations . . . 81
9.5 Future work . . . 83
9.5.1 Dataset and annotation . . . 83
9.5.2 Segmentation and model deployment . . . 83
9.5.3 Geometric extraction . . . 83
9.5.4 System integration and field validation . . . 84
9.6 Closing remarks . . . 84

Bibliography 85

List of Figures

2.1 Agricultural inter-row environment, vehicle frames, and geometric error definition 6
2.2 Complete vision-based local guidance pipeline from camera stream to actuator commands 7

4.1 Fully Convolutional Network (FCN) architecture with progressive skip connections combining deep semantic features with shallow appearance representations (adapted from Long et al. [9]). 22
4.2 DeepLabV3 architecture featuring the Atrous Spatial Pyramid Pooling (ASPP) module with parallel atrous convolutions and image-level pooling (adapted from Chen et al. [10]). 24
4.3 MobileNetV3 and Lite Reduced ASPP (LR-ASPP) architecture: (a) inverted residual block combining depthwise separable convolution, linear bottleneck, and Squeeze-and-Excitation channel attention; (b) segmentation head combining stride-16 features modulated by global context gating with stride-8 fine features (adapted from Howard et al. [11]). 29
4.4 Original Segment Anything Model (SAM) architecture with ViT image encoder, prompt encoder, and mask decoder (adapted from Kirillov et al. [12]). SAM 3 extends this foundation with a dual detector-tracker architecture and open-vocabulary concept segmentation. 31

5.1 Sample field images across crops and corresponding training masks 34
5.2 SAM 3 zero-shot autolabeling and mask composition workflow . . . 36
5.3 Training and validation loss per epoch of FCN, DeepLabV3, and LR-ASPP on the manual lettuce dataset. The best checkpoint is the epoch with the minimum validation loss. The backbone is unfrozen after epoch 10. 41

6.1 Horizontal strip decomposition and column-wise projection for path interval detection . . . 48
6.2 Reweighted line fitting of corridor boundaries and centerline construction . . . 49

7.1 Physical mobile robot platform and front camera mounting setup . 58
7.2 ROS 2 node graph on the robot, as shown by `rqt_graph`. Ellipses are nodes and rectangles are topics. . . . 60

8.1 Visual comparison of segmentation outputs on lettuce test images and on tomato and cucumber images . . . 69
8.2 Comparative evaluation of geometric reference extraction across the four evaluated algorithms . . . 71
8.3 Lateral tracking error and heading error profiles during inter-row autonomous navigation . . . 75

Chapter 1

Introduction

▶ **DA COMPLETARE: Contesto di tesi e motivazione applicativa**

Inquadrare l'attività svolta nell'ambito della tesi di laurea magistrale:

- Collaborazione con il gruppo di ricerca di riferimento (Politecnico di Torino).
- Esigenze dell'agricoltura di precisione: diserbo meccanico selettivo, monitoraggio delle colture e riduzione dell'impatto ambientale senza dipendenza da segnali GNSS RTK sotto chioma o in serra.
- Obiettivo di realizzare una soluzione autonoma, modulare e fruibile su calcolatori embedded a basso costo (famiglia NVIDIA Jetson).

1.1 Context and motivation

Global population growth and rising food demand exert increasing pressure on agricultural productivity, while the sector faces persistent labor shortages and rising operating costs [1, 2]. Addressing these demands requires higher automation in field operations such as weeding, spraying, and crop monitoring. Row-based crops are among the most studied applications of agricultural robots and account for about 75% of the planted cropland in the United States [3]. Autonomous ground vehicles designed for these settings must navigate along inter-row corridors reliably without damaging crops or compacting productive soil beds.

Outdoor agricultural navigation historically relied on Global Navigation Satellite Systems (GNSS) augmented by Real-Time Kinematic (RTK) corrections. However, satellite positioning presents well-documented limitations in field environments: thick crop canopies cause multi-path reflections and severe signal attenuation, while greenhouse installations obstruct satellite reception entirely [1, 3]. Furthermore, RTK base station infrastructure adds cost. Alternative exteroceptive sensors such as 3D LiDAR provide direct spatial measurements, but their high unit cost hinders

economic scalability across commercial robotic fleets, and their point clouds lack the color and textural information required to distinguish crops from weeds [1, 3]. Monocular RGB cameras offer a cost-effective alternative that captures detailed chromatic and texture cues directly from the scene.

Early vision-based guidance pipelines extracted navigation lines through handcrafted image processing techniques. These approaches compute color vegetation indices to isolate plant matter from soil, apply intensity thresholding such as Otsu's method, and detect corridor edges using line-fitting algorithms or Hough transforms [1, 4]. Although computationally lightweight, these calibrated pipelines degrade when ambient field conditions shift. Variable sunlight angles, cast shadows from foliage, soil moisture gradients, specular reflections on wet leaves, and weed infestations frequently disrupt fixed thresholding rules and generate false boundary estimates.

Deep learning architectures, particularly convolutional neural networks for semantic segmentation, overcome these hand-tuning limitations. By learning hierarchical visual representations directly from image datasets, segmentation networks achieve higher tolerance to illumination shifts, ground irregularities, and weed clutter than handcrafted color models.

However, deep neural networks introduce a practical operational bottleneck: supervised training requires extensive pixel-level ground truth. Manually labeling agricultural scenes is labor-intensive and must cover multiple crop types, growth stages, and ambient conditions. In addition, running neural networks on compact onboard computers requires balancing segmentation accuracy against processing speed and power consumption, as steering commands must be updated fast enough to guide the vehicle in real time.

This thesis addresses these challenges through a modular pipeline that combines automatic dataset annotation with SAM 3, real-time onboard semantic segmentation, and deterministic geometric reference extraction for local robot guidance.

1.2 Objectives of the thesis

The main objectives of this work are:

- Evaluate whether segmentation networks trained on SAM 3 labels, used without manual correction, approach a network trained on manual masks, using manually annotated lettuce images as ground truth.
- Select a segmentation architecture balancing accuracy and inference speed for robotic deployment, through offline evaluation of FCN, DeepLabV3, and LR-ASPP on manually annotated lettuce images.

- Evaluate the effect of output label configurations (four-class, three-class, and binary path) on path segmentation accuracy.
- Implement an edge-based geometric extractor that converts segmentation masks into local image-space navigation references, and compare it with three baseline algorithms.
- Integrate the selected perception and geometric components on a robot platform for field validation.

1.3 Organization of the work

The thesis is organized as follows:

Chapter 2 introduces the agricultural navigation problem, the role of vision-based local guidance, and the segmentation-to-control pipeline structure.

Chapter 3 reviews vision-based methods for agricultural row navigation, covering classical and learning-based segmentation approaches, geometric extraction techniques, and experimental validation in related work.

Chapter 4 details the mathematical formulations and designs of the deep learning architectures evaluated in this work, focusing on FCN, DeepLabV3, MobileNetV3 with the LR-ASPP head, and SAM 3.

Chapter 5 describes dataset preparation with manual annotation and automatic SAM 3 labeling, empirical comparisons among the three architectures on lettuce, and evaluations of supervision strategies across label configurations.

Chapter 6 presents the geometric extractor that converts corridor masks into navigation references through horizontal-strip interval selection and independent edge fitting, together with three baseline algorithms used for comparison.

Chapter 7 describes the robot hardware, camera configuration, software integration using ROS, and the deployment of the trained segmentation model.

Chapter 8 reports offline and field validation results for segmentation accuracy, geometric reference extraction, and closed-loop navigation performance.

Chapter 9 summarizes the main findings, discusses limitations, and identifies directions for future work.

Chapter 2

Background and Problem Formulation

This chapter establishes the agricultural navigation context and formulates the vision-based local guidance problem addressed in this thesis.

2.1 Agricultural autonomous navigation

2.1.1 Navigation tasks and requirements

Agricultural robots perform tasks such as monitoring, spraying, harvesting, and weeding in structured crop environments [1]. Navigation between crop rows requires the robot to follow a path aligned with the row structure while avoiding plants and obstacles. The guidance reference must be computed from local sensing, as global GNSS positioning may be obstructed by crop canopies or greenhouse structures [1, 3].

Vision-based methods identify crop rows, inter-row paths, or crop boundaries from camera images and convert these observations into steering commands [1, 2]. The task is local: the robot maintains its position relative to the visible row structure, not a global map.

2.1.2 Environmental challenges

Field conditions vary with illumination, shadows, weather, growth stage, and weed presence [5, 2]. Plants may occlude parts of the path, and gaps or irregular spacing disrupt row structure. These changes affect both the appearance of image features and the reliability of geometric references extracted from them.

Segmentation models trained on one set of conditions must generalize to new lighting, crops, or row configurations. Geometric extractors must handle incomplete or ambiguous corridor observations without assuming perfect segmentation [6].

2.2 Vision-based local guidance

2.2.1 Sensing modalities

Camera-based perception provides appearance information for distinguishing plants, soil, and paths. Monocular RGB cameras are simpler and less expensive than LiDAR or stereo systems, but do not directly measure depth [1]. While RGB-D cameras provide both appearance and depth measurements, their performance depends on field conditions and hardware cost constraints.

This thesis uses monocular RGB sensing for local guidance. It does not reconstruct three-dimensional field structure or maintain a global map.

2.2.2 Image-space reference extraction

The guidance pipeline converts image observations into a reference suitable for control. Depending on the method, this may be a horizontal position, a fitted line, or a curve [7, 6, 4]. Image-space references describe the path geometry in pixel coordinates; they must be interpreted by the controller to generate velocity commands.

Straight-line models assume locally linear boundaries and produce simple references for proportional control [4]. They do not capture distant curves or complex row patterns, but are computationally efficient and suitable for short prediction horizons.

▶ DA COMPLETARE: Modello cinematico 4WS e formulazione analitica degli errori

Integrare la formalizzazione matematica del veicolo e del controllo:

- Modello cinematico per piattaforma 4WS a sterzatura simmetrica in controfase ($\delta_f = -\delta_r$) e relazione cinematica: $\dot{x} = v\cos\theta$, $\dot{y} = v\sin\theta$, $\dot{\theta} = \omega = \frac{2v\tan\delta}{L}$.
- Trasformazione rigida e proiettiva tra coordinate pixel (u, v) nel piano immagine e coordinate metriche nel frame robot $\{R\}$.
- Definizione rigorosa dell'offset laterale e_y e dell'heading error e_θ calcolati rispetto alla retta di mezzeria stimata.
- Formulazione della legge di guida proporzionale / Stanley per la determinazione della velocità angolare $\omega = K_p e_y + K_\theta e_\theta$.



Figure 2.1: Agricultural inter-row environment, vehicle frames, and geometric error definition

2.3 The segmentation-to-control pipeline

[SPAZIO FIGURA / SCHEMA]

Complete vision-based local guidance pipeline from camera stream to actuator commands

Schema a blocchi dell'architettura di navigazione: flusso continuo dall'acquisizione delle immagini monoculari, attraverso la rete di segmentazione semantica, l'estrazione geometrica dei bordi del corridoio, il calcolo degli errori di guida e la generazione dei comandi di velocità (v, ω).

Figure 2.2: Complete vision-based local guidance pipeline from camera stream to actuator commands

2.3.1 Perception stage

The perception stage assigns semantic labels to image pixels. For inter-row navigation, target classes typically comprise crop plants, the traversable path, covering material such as plastic mulch, and background [6]. The definition of output classes directly determines what the geometric stage can extract: a binary vegetation mask identifies crop rows, whereas a path mask isolates the drivable space between them [2, 3].

Segmentation models are trained on annotated field images. While transfer learning from pretrained backbones reduces training duration and improves feature generalization, the classification head must be tailored to the specific target crop classes and deployed at low latency on embedded hardware [3].

2.3.2 Geometric extraction stage

The geometric stage reduces the dense segmentation mask to a low-dimensional guidance reference for robot steering. Existing methods differ in their spatial search strategy and fitted geometric model: some aggregate mask pixels across image columns [4, 3], whereas others extract local intervals from horizontal strips [6] or search for line candidates within bounded angular regions [7].

The extraction algorithm must tolerate segmentation noise, missing plants, and cast shadows without producing erratic steering targets. Validation checks can reject estimates that fail geometric consistency thresholds, requiring the controller to handle transient frames where no valid reference is recovered [5].

2.3.3 Control integration

The controller converts the extracted geometric reference into forward velocity and steering commands. Proportional and look-ahead laws regulate steering using the lateral displacement and heading orientation errors relative to the image center [4].

In field operations, raw error signals extracted from semantic segmentation exhibit high-frequency jitter and occasional impulse disturbances caused by irregular foliage contours or transient detection dropouts. Directly feeding unfiltered errors into the steering loop induces actuator chattering and abrupt steering adjustments that compromise vehicle stability. Consequently, the control interface incorporates error conditioning structured across three complementary mechanisms:

1. **High-frequency smoothing:** a recursive low-pass filter attenuates persistent image-space tracking jitter without introducing excessive phase delay during row transitions;

2. **Impulse outlier rejection:** large step variations between consecutive frames are evaluated over temporal windows to isolate single-frame false detections while preserving genuine trajectory adjustments;

3. **Temporal hold and deadband:** a zero-order hold maintains command continuity across short-term perception dropouts, while a symmetric deadband around the corridor center prevents steering hunting.

The mathematical formulation and parameter tuning of this conditioning pipeline on the Demetra robotic platform are detailed in Section 7.2.1 of Chapter 7.

Closed-loop field evaluation measures the robot's ability to maintain row alignment under operational conditions. High segmentation accuracy and accurate geometric fitting are necessary prerequisites, but stable inter-row tracking ultimately depends on real-time execution and disturbance rejection over continuous runs [4, 2, 3].

Chapter 3

State of the Art

This chapter reviews vision-based methods for agricultural row navigation, focusing on the conversion of image observations into a local guidance reference. As surveyed by Bai et al. [1], vision sensors provide rich appearance and geometry cues for autonomous field guidance at low hardware cost. The review covers visual perception, geometric extraction, and the integration of these stages with robot control. Dataset preparation and the implementation used in this thesis are described in Chapters 5 and 6.

3.1 Classical image-processing methods

Classical pipelines combine image filtering, thresholding, and application-specific geometric rules. Mendez et al. convert vineyard images to grayscale, apply Gaussian filtering and Otsu thresholding, select the largest blob, and use column pixel counts to estimate the row center [4, Section 3.4]. Such methods can be effective in the conditions for which their parameters were selected, but changes in illumination, weeds, and crop appearance can require retuning [1].

3.2 Learning-based segmentation for agricultural navigation

3.2.1 Semantic segmentation approaches

Semantic segmentation assigns a class to each pixel, but the label definition determines what the geometric stage can recover. De Silva et al. train U-Net to predict narrow crop-row masks in sugar-beet images. Their labels represent row structure rather than individual plant pixels or traversable soil [2, Sections 3.3 and 4.1]. Navone et al. instead predict a binary vegetation mask from RGB images

using a MobileNetV3 backbone and an LR-ASPP head, then combine that output with depth for reference extraction [8, 3].

Like other path-oriented approaches in the literature, this work segments the traversable soil corridor between crop rows rather than the plants themselves. The segmentation model provides a candidate region of drivable ground, and the downstream geometric stage then extracts the lateral boundaries and computes the steering reference.

3.2.2 Training data and generalization

Pixel-level annotation is time-consuming, and agricultural datasets must represent changes in crop, growth stage, and lighting. De Silva et al. include 11 field variations in their dataset [2, Section 3]. Navone et al. initialize their segmentation model with ImageNet weights and train using synthetic AgriSeg images supplemented by 100 real crop images [3, Section 4.1, Table 1]. Pretraining and the choice of training images are separate design choices.

3.3 Semantic segmentation architectures

3.3.1 FCN, DeepLabV3, and mobile architectures

Fully Convolutional Networks (FCN) replace fully connected layers with convolutions to produce dense pixel-level predictions [9]. DeepLabV3 incorporates atrous convolutions and atrous spatial pyramid pooling to capture multi-scale context without decreasing feature resolution [10]. For embedded applications where compute resources are constrained, Howard et al. [11] introduced MobileNetV3 together with the Lite Reduced Atrous Spatial Pyramid Pooling (LR-ASPP) head, which reduces latency by using depthwise separable convolutions.

The choice of architecture affects both segmentation accuracy and inference speed. Deeper backbones or larger receptive fields can yield higher IoU values at the cost of additional compute. For robotic systems operating in closed loop, inference latency on edge hardware dictates the achievable control frequency.

3.3.2 Transfer learning and pretraining

Transfer learning initializes network weights from models pretrained on large visual corpora such as ImageNet. The pretrained backbone is a generic feature extractor, while the task-specific segmentation head is adapted to produce class scores matching the target dataset. In agricultural vision, pretraining reduces training time and mitigates overfitting when domain-specific annotated datasets are limited in size. For instance, Li et al. [6, Section 2.2.2] demonstrate that transferring

weights learned on corn fields to other crops (such as tomatoes, cucumbers, and wheat) preserves segmentation accuracy with small training sets. Similarly, Navone et al. [3, Section 4.1] initialize their segmentation backbone with ImageNet weights prior to fine-tuning on tree-row imagery.

The extent of pretraining differs across implementations. Some setups retain pretrained segmentation heads and replace only the final classification layer, whereas others initialize only the backbone and train the segmentation head from scratch.

3.4 Dataset preparation and model-assisted annotation

3.4.1 Manual annotation challenges

Manual pixel-level annotation requires annotators to label every pixel in training images. For agricultural datasets spanning multiple crops, growth stages, and lighting conditions, this process is time-consuming and expensive. Annotation consistency depends on annotator interpretation of ambiguous boundaries, particularly where plants overlap soil or covering material.

3.4.2 SAM and foundation models for segmentation

Foundation models for segmentation have established model-assisted annotation as an effective alternative to purely manual labeling. The Segment Anything project built its dataset with a model-in-the-loop data engine in three stages (assisted-manual, semi-automatic, and fully automatic), obtaining more than one billion masks on 11 million images [12]. While early foundation models required spatial prompts such as bounding boxes or foreground/background clicks for every target instance, Segment Anything Model 3 (SAM 3) introduces open-vocabulary concept segmentation driven directly by textual descriptions and visual exemplars [13].

In precision agriculture, model-assisted annotation with foundation models has been explored primarily for satellite-based field boundary delineation and canopy phenotyping. However, its systematic application to agricultural ground navigation remains largely unexplored. In this thesis, SAM 3 is employed offline as a zero-shot concept annotator, and its masks are used without manual correction. On lettuce, networks trained on SAM 3 labels with different class sets (path, plants, plastic mulch) are compared against manual ground truth; binary path labels generated for the complete lettuce, tomato, and cucumber collection train the network deployed on the robot. This offline annotation pipeline decouples the heavy computational burden of foundation models from the real-time inference requirements of the onboard robot controller.

3.5 Geometric extraction methods

The geometric stage reduces a mask to a quantity the controller can use. Depending on the representation and method, the output may be a horizontal image position, a straight line, a crop-edge estimate, or a curve. These outputs describe different tasks and coordinate systems.

3.5.1 Line fitting and crop-edge detection

Chen et al. estimate a path line for greenhouse cucumber rows from navigation points selected in plant–soil segmentation. Their prediction-point Hough transform uses a least-squares estimate to restrict the line search and is compared with traditional Hough and least-squares fitting [7, Sections 2–3].

Kneip et al. detect crop–ground transitions in scanlines of a stereo elevation map, then fit a linear crop-edge model using a Huber M-estimator and iteratively reweighted least squares. Their output is a crop boundary in ground-plane coordinates, not an image-space estimate of the inter-row path [5, Sections 6–7].

3.5.2 Column-wise histogram methods

Histogram methods aggregate mask pixels by image column. Mendez et al. estimate the row center from the five columns with the largest pixel counts in a thresholded background region [4, Section 3.4.3]. Navone et al. use a different input: a temporally combined vegetation mask refined with depth. SegMin smooths its column histogram and selects the minimum; SegMinD weights the vegetation mask by normalized inverse depth before calculating the histogram [3, Section 3]. In both cases the output is a horizontal position, not a full path line.

3.5.3 Horizontal strips and adaptive multi-ROI methods

Li et al. combine a Faster-U-net inter-row-path mask with an adaptive multi-ROI procedure. Horizontal regions supply ordered local points, which are used as control points for a multisegment cubic B-spline. Their experiments use 40-pixel search regions and discuss the trade-off between the number of points and fitting behavior [6, Section 2.2.3, Figure 9].

This is the closest geometric comparison for the present work because both pipelines select local evidence from horizontal image regions. The representation and fitted model differ: Li et al. use an inter-row-path mask and a spline, whereas this thesis estimates a local straight reference from the soil corridor between parallel crop rows.

3.5.4 Triangle Scan Method

De Silva et al. propose the Triangle Scan Method (TSM) for crop-row masks. It first scans an upper horizontal band to find an anchor column, using a preset anchor if the scan fails a threshold. Candidate lines from that anchor to positions along the base of a triangular region are then evaluated; the line with the greatest mask support defines the central crop row. Its two endpoints provide position and angular errors for visual servoing [2, Section 4.2, Algorithm 1].

TSM is not the same as extracting one point from each of several horizontal strips: the upper scan establishes an anchor for a bounded line search. It also operates on crop-row masks, so its output is not directly equivalent to a reference extracted from a soil mask.

3.5.5 Bilateral boundary tracking versus centroid reduction

In agricultural vision pipelines that convert binary segmentations into geometric references, existing methods fall into distinct categories, each characterized by specific operational trade-offs:

1. **Crop-canopy feature extraction with RANSAC:** A classical paradigm detects feature points along crop canopies or plant stalks, partitions them into left and right row clusters, and fits linear models using Random Sample Consensus (RANSAC) combined with ordinary least squares [1]. Recent monocular vision implementations extract crop row anchor points via DBSCAN clustering followed by RANSAC fitting to generate navigation lines, with reported angular and lateral errors minimized on the central crop row [14]. While functional in dense, uniform vegetation, this approach depends on continuous plant growth and degrades when seedlings are missing or canopy density is irregular. In addition, RANSAC is non-deterministic: random hypothesis generation introduces frame-to-frame angular jitter between consecutive video frames, requiring downstream temporal filtering to stabilize steering commands. The stochastic nature of RANSAC stems from its iterative random sampling strategy: each iteration selects a minimal subset of points (typically two for line fitting), computes a candidate model, and counts inliers within a residual threshold. To achieve 99% confidence of finding an all-inlier sample for a two-point linear model when outliers constitute 50% of the data, the algorithm requires 16 iterations, rising to 35 iterations when the outlier ratio reaches 65%; this sample count grows exponentially as outlier contamination increases [15]. On embedded platforms operating at 20 Hz with strict latency budgets, the computational overhead and output variance of RANSAC present practical limitations for closed-loop control.

2. **Global contour tracking with line fitting:** An alternative approach applies edge operators (such as Canny filtering) or boundary tracing algorithms (`findContours`) over the full segmented corridor mask, separating the resulting pixel cloud into lateral boundaries before fitting lines with RANSAC. On dense and noisy boundary masks, this introduces variable computational latency and retains the stochastic jitter inherent to random sampling, confirming that raw RANSAC output on monocular imagery lacks the determinism required for smooth steering commands.

3. **Horizontal strip decomposition with centroid reduction:** Methods operating on segmented drivable soil corridors (such as the multi-ROI scheme of Li et al. [6]) partition the image into horizontal bands and extract the bounding interval $[x_L, x_R]$ within each strip. However, these methods immediately collapse each interval into its midpoint:

$$x_C = \frac{x_L + x_R}{2} \tag{3.1}$$

   fitting curves (such as cubic B-splines or polynomials) directly to the sequence of centroids x_C. This premature reduction discards the distinction between the two physical edges. Under asymmetric disturbances—such as protruding leaves, localized weeds, or shadows extending along one side of the corridor—the midpoint shifts toward the intrusion, which introduces systematic bias in both lateral offset and heading angle.

4. **Cascaded deep learning for line extraction:** Recent studies investigate two-stage deep architectures, where an initial segmentation model is followed by a secondary network dedicated to semantic line detection. While capable of learning line features directly from data, running two cascaded neural networks on embedded platforms exceeds the memory and latency limits required for closed-loop robotic control (≥ 20 Hz).

5. **Single-edge elevation fitting:** M-estimators have also been applied to crop boundaries, such as Huber-loss Iteratively Reweighted Least Squares (IRLS) on stereo elevation scanlines [5]. However, these formulations track a single cutting boundary along dense harvesting fronts, rather than parallel crop boundaries defining an inter-row corridor from monocular RGB imagery.

To address the trade-off between computational determinism and disturbance decoupling without cascading neural networks, this thesis investigates a decoupled dual-edge linear formulation:

$$x_L(y) = m_L y + q_L, \qquad x_R(y) = m_R y + q_R \tag{3.2}$$

Both boundaries are estimated by Iteratively Reweighted Least Squares (IRLS) with the weight rule

$$w_i = \frac{1}{1 + \left(\frac{r_i}{\sigma}\right)^2} \tag{3.3}$$

applied for a fixed number of four iterations to at most one boundary sample per horizontal strip ($N = 16$ on the robot); Chapter 6 details how `numpy.polyfit` applies these weights. A perturbation on one boundary changes only the weights and fit of that boundary. The centerline reference $x_C(y) = (x_L(y) + x_R(y))/2$ is computed after both boundary fits.

3.5.6 Comparison of geometric representations

Table 3.1 summarizes the input representation and geometric output of the methods discussed above.

Table 3.1: Perceptual inputs and geometric references in selected agricultural navigation studies.

Study	Perceptual input	Geometric operation	Output
Chen et al. [7]	Plant–soil segmentation and navigation points	Restricted Hough search around a predicted point	Image-space path line
Kneip et al. [5]	Stereo elevation map	Scanline transitions and IRLS linear fit	Ground-plane crop edge
Mendez et al. [4]	Thresholded background region	Column counts and mean of the five largest	Horizontal row-center position
Navone et al. [3]	Vegetation mask with depth	Smoothed column histogram; inverse-depth weighting in SegMinD	Histogram-minimum position
Li et al. [6]	Predicted inter-row-path mask	Adaptive horizontal regions and cubic B-spline	Curved image-space path
De Silva et al. [2]	Predicted crop-row mask	Upper anchor and triangular line search	Central crop-row line
This thesis (Proposed DELE)	Predicted soil-corridor / mulch mask	Discrete horizontal strips, separate IRLS fits of the two boundaries	Straight image-space centerline $x_C(y)$ and (e_y, e_θ)

The methods are not directly ranked by this comparison: their inputs and outputs differ. Li et al. provide the closest structural precedent for strip-based point selection, while TSM illustrates a line search for a crop-row representation.

Neither method uses the same soil-corridor mask and straight-line model as the pipeline developed in this thesis.

3.6 System integration and experimental validation

3.6.1 Perception and control integration

Navigation systems use the extracted reference to generate motion commands. Navone et al. convert the displacement of their histogram reference into linear and angular velocity commands and smooth them with an Exponential Moving Average [3, Section 3.4]. Mendez et al. use proportional control based on row-center displacement and the distribution of background pixels [4, Section 3.5]. De Silva et al. combine the central row's angular and positional errors in a proportional visual-servoing controller for simulation [2, Section 4.3].

3.6.2 Scope of reported evaluations

The studies evaluate different stages of navigation. Kneip et al. assess crop-edge detection on recorded harvesting data [5, Section 8]; Chen et al. report angular errors and processing times for image-based path fitting [7, Sections 3.6, Tables 6–7]. De Silva et al. evaluate row detection on real images and visual servoing in simulation, with preliminary real-world row following also reported [2, Sections 5.2–5.3 and 6, Table 5]. Mendez et al. report closed-loop vineyard trials, measuring centering error from the distances to the two sides of the row [4, Sections 3.6 and 4]. Navone et al. test in simulation and crop fields, using LiDAR and attitude measurements to assess displacement and heading [3, Sections 4.3–4.4].

Segmentation IoU, image-space line error, field tracking error, and processing time measure different parts of the system. Results from these studies therefore provide context, not a common performance benchmark.

3.7 Positioning of this thesis

The system developed in this thesis addresses autonomous navigation in GNSS-denied crop rows using monocular RGB vision and embedded computation. Its technical positioning relative to the state of the art rests on three architectural decisions:

1. **Perceptual representation (Soil corridor versus canopy):** Rather than detecting crop stalks or foliage masks, the segmentation stage targets the

navigable soil corridor (and plastic mulch). This makes the navigation reference independent of crop growth stage, canopy height, and missing plants (fallanze), providing a continuous geometric guide directly at ground level.

2. **Geometric extraction (Decoupled deterministic boundaries versus centroid reduction):** Rather than collapsing strip intervals into midpoints prior to fitting (as in Li et al. [6]) or employing stochastic RANSAC over dense edge contours, the proposed pipeline decouples the left and right physical corridor boundaries. Each boundary is fitted independently by iteratively reweighted least squares with a fixed number of four iterations. A disturbance on one side (such as a leaf overhang or a shadow) enters only the fit of that boundary; the centerline, as the mean of the two edge lines, receives half of that boundary's deviation.

3. **Embedded deployment feasibility:** The architecture decouples offline automatic dataset generation from onboard execution. Large foundation models (SAM 3) are restricted to offline zero-shot concept annotation, while real-time inference on the NVIDIA Jetson AGX Orin is handled by a lightweight LR-ASPP network (using a MobileNetV3 backbone) paired with the deterministic IRLS geometric extractor. The entire pipeline executes in closed loop at 20 Hz without requiring secondary neural networks, stereo elevation mapping, or LiDAR sensors.

The chapters that follow detail and evaluate each stage of this pipeline: Chapter 4 formulates the deep learning architectures, Chapter 5 analyzes the annotation protocols and comparative model training, Chapter 6 formalizes the geometric extractors, Chapter 7 describes the Demetra robotic platform and onboard ROS integration, and Chapter 8 reports offline benchmarks and field tests.

Chapter 4

Deep Learning Architectures for Semantic Segmentation

In computer vision, semantic segmentation is the task of classifying every pixel in an image according to the category of the surface or object it belongs to [9], [16, Chapter 12]. Unlike image-level classification, which predicts a single global label for the entire scene, or object detection, which localizes entities with coarse bounding boxes, semantic segmentation generates a dense, pixel-level partition of the scene.

In this thesis, autonomous row navigation relies on a monocular vision pipeline in which semantic segmentation is the only perception stage. Rather than relying on satellite positioning (RTK-GPS) or active range sensors (LiDAR), the navigation system extracts the vehicle guidance reference directly from single RGB images by segmenting the navigable soil corridor from surrounding crop canopies, weed patches, and plastic mulch films under variable outdoor illumination.

To investigate the trade-off between segmentation accuracy, model complexity, and real-time execution on the mobile robot, this chapter analyzes three distinct deep learning architectures:

- **Fully Convolutional Networks (FCN):** pairing a dilated ResNet-50 backbone with a convolutional segmentation head (`FCNHead`);
- **DeepLabV3:** pairing a dilated ResNet-50 backbone with an Atrous Spatial Pyramid Pooling (ASPP) head;
- **MobileNetV3 with LR-ASPP:** pairing a MobileNetV3-Large backbone with a Lite Reduced ASPP head.

In addition, the Segment Anything Model 3 (SAM 3) is introduced as an offline foundation model for semi-automated training dataset annotation.

The remainder of this chapter is structured as follows. Section 4.1 establishes the mathematical formulation and encoder-decoder principles. Sections 4.2, 4.3, and 4.4 detail the three segmentation networks, while Section 4.5 describes the SAM 3 annotation pipeline.

4.1 Problem formulation and encoder-decoder principles

Let $I \in \mathbb{R}^{H\times W\times 3}$ denote an input color image with height H and width W. Semantic segmentation aims to estimate a dense label map $Y \in \{1,\dots,C\}^{H\times W}$, where each pixel (i,j) is assigned to one of C mutually exclusive semantic classes [9], [16, Chapter 12]. Deep segmentation networks approximate this mapping through a parameterized function $f_\theta(I)$ that outputs an unnormalized score tensor $Z \in \mathbb{R}^{H\times W\times C}$ containing the class logits for every pixel location. The predicted probability of class c at pixel (i,j) is obtained by applying a pixel-wise softmax over the class dimension:

$$p_{i,j,c} = \frac{\exp(Z_{i,j,c})}{\sum_{k=1}^{C}\exp(Z_{i,j,k})}. \tag{4.1}$$

During supervised training, network parameters θ are optimized by minimizing a loss function $\mathcal{L}(Y^*,p)$ that quantifies the discrepancy between the predicted probability map p and the ground-truth annotations $Y^* \in \{0,1\}^{H\times W\times C}$, represented in one-hot form [9]. Through backpropagation, the gradients of this loss with respect to θ guide stochastic optimization algorithms (such as SGD or AdamW) to iteratively update the network weights. Because semantic segmentation operates densely across the spatial grid, the overall loss is evaluated by aggregating individual prediction errors over all pixel locations (i,j) [9], [16, Chapter 12].

A widely used training objective for semantic segmentation is the pixel-wise categorical cross-entropy loss, averaged across all pixels:

$$\mathcal{L}_{\mathrm{CE}}(Y^*,p) = -\frac{1}{HW}\sum_{i=1}^{H}\sum_{j=1}^{W}\sum_{c=1}^{C} Y^*_{i,j,c}\log(p_{i,j,c}). \tag{4.2}$$

For each pixel, this formulation penalizes the negative logarithm of the probability assigned to the ground-truth category: the penalty approaches zero when the network predicts the correct class with high confidence ($p_{i,j,c} \approx 1$), and grows large when the assigned probability is low ($p_{i,j,c} \to 0$).

In practical segmentation tasks, loss functions can be adapted to handle specific domain challenges. When target classes occupy highly disproportionate image areas, cross-entropy can be extended with class-specific penalty weights w_c to mitigate the dominance of majority classes during optimization [9], as formulated for our

agricultural dataset in Chapter 5. Other common alternatives in the literature include region-based losses (such as the Dice loss) that optimize spatial segment overlap, and boundary-aware losses designed to refine object contours.

The output stride S denotes the ratio of the input image spatial resolution to the spatial dimensions of the extracted feature map [10]. Many classification backbones progressively reduce spatial resolution through consecutive strided convolutions and pooling operations, reaching an output stride of up to $S = 2^5 = 32$ [16, Chapter 12]. While spatial downsampling expands the effective receptive field and enables deeper layers to extract abstract semantic features (determining *what* is depicted in the scene), it discards fine-grained spatial coordinates (obscuring *where* object boundaries lie at the pixel level). Segmentation backbones may instead employ smaller output strides, such as $S = 8$ or $S = 16$, using atrous convolutions to preserve denser feature representations [10].

To resolve this trade-off between high-level semantic abstraction and pixel-level spatial precision, modern semantic segmentation models commonly adopt an encoder-decoder architecture:

- **Encoder (Backbone):** A sequence of feature-extraction stages that produces high-level semantic representations, yielding a feature tensor $F_{\text{enc}} \in \mathbb{R}^{\frac{H}{S} \times \frac{W}{S} \times C_{\text{enc}}}$, where S denotes the encoder output stride. Depending on the architecture, intermediate feature maps at different resolutions may also be supplied to the decoder.
- **Decoder and segmentation head:** The decoder processes the encoded features and progressively recovers spatial resolution, optionally combining them with higher-resolution encoder features through skip connections [9]. A final segmentation head maps the resulting features to class logits, which are upsampled to the input image resolution when necessary.

4.2 Fully Convolutional Networks (FCN)

Traditional Convolutional Neural Networks are designed for image-level classification, producing a single label for the entire input. Fully Convolutional Networks (FCN), introduced by Long et al. [9], adapt these architectures for semantic segmentation by predicting a class label for every individual pixel in an end-to-end manner.

This is achieved through convolutionalization: replacing the final dense (fully connected) layers with convolutional filters allows the network to process images of arbitrary size and output dense spatial score maps. In this study, the FCN architecture is instantiated with a ResNet-50 [17] residual backbone and an `FCNHead` segmentation head, matching the standard TorchVision implementation (`fcn_resnet50`).

4.2.1 Convolutionalization of classification networks

In standard classification networks, the feature map produced by the final convolutional stage is flattened into a one-dimensional vector and fed into dense (fully connected) layers. Because a dense layer contains a fixed number of weights dedicated to each flattened input position, the network is restricted to a rigid, predetermined input image resolution.

As detailed by Géron [16, Chapter 12], a dense layer with C units connected to an input feature map of spatial dimension $h \times w$ with C_{in} channels can be replaced by a convolutional layer with C filters, a filter size equal to $h \times w$, and *valid* padding. The convolutional layer then outputs a $1 \times 1 \times C$ tensor containing the same values produced by the dense layer.

Because a convolution is applied at every spatial position, the convolutionalized network no longer requires a fixed input size. When the input feature map is larger than $h \times w$, the same filters slide over it and output a spatial grid of class scores in a single forward pass. This result is equivalent to sliding the original classification network across the image, but each intermediate activation is computed only once instead of once per overlapping window [16, Chapter 12].

4.2.2 Skip architecture and upsampling

The classification networks adapted by Long et al. (AlexNet, VGG-16, and GoogLeNet [9]) apply an overall stride of $S = 32$, so the final score map is 32 times smaller than the input image [16, Chapter 12]. Bilinear interpolation works reasonably well only for upsampling factors up to $4\times$ or $8\times$ [16, Chapter 12]; a single $32\times$ interpolation step therefore cannot restore boundary detail.

For in-network upsampling, Long et al. [9] adopted transposed convolutions (also referred to as fractionally strided convolutions or deconvolutions, available in PyTorch as `nn.ConvTranspose2d`). A transposed convolution with stride s can be interpreted as expanding the input feature map by inserting $s - 1$ rows and columns of zeros between adjacent elements, followed by a standard convolution with filter weights $w \in \mathbb{R}^{K \times K}$ [16, Chapter 12]. Algebraically, the operation reverses the forward and backward passes of a convolution [9]. In the original FCN, the intermediate $2\times$ upsampling layers are initialized to bilinear interpolation and then learned, whereas the final upsampling layer is kept fixed to bilinear interpolation [9].

Upsampling directly from stride 32 in a single step (FCN-32s) yields coarse segmentation boundaries. To refine them, Long et al. introduced skip connections that fuse high-level semantic features from deep layers with fine-grained appearance information from shallower layers [9]:

- **FCN-32s:** Upsamples the final stride-32 score map directly to image resolution in a single $32\times$ step.

- **FCN-16s:** Upsamples the stride-32 score map by a factor of 2×, adds it element-wise to predictions computed from the stride-16 layer (`pool4` in VGG-16), and upsamples the combined map by 16×.
- **FCN-8s:** Upsamples the fused FCN-16s score map by 2×, sums it with predictions from the stride-8 layer (`pool3`), and upsamples the result by 8× back to $H \times W$.

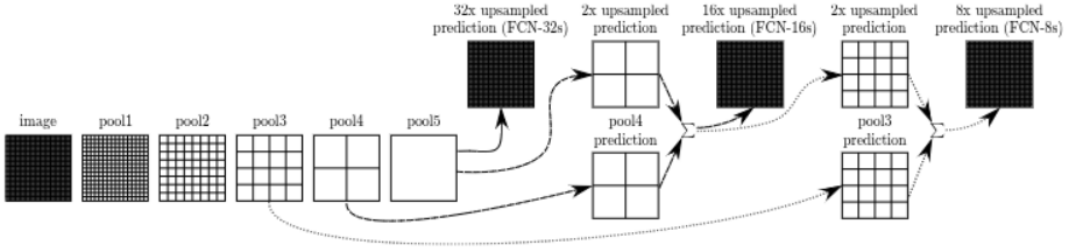


Figure 4.1: Fully Convolutional Network (FCN) architecture with progressive skip connections combining deep semantic features with shallow appearance representations (adapted from Long et al. [9]).

Implementation note: The experimental evaluation in this thesis employs the TorchVision `fcn_resnet50` implementation [18]. Unlike the original FCN-8s architecture, this model does not fuse predictions from multiple backbone stages. Instead, its ResNet-50 backbone replaces the spatial downsampling in `layer3` and `layer4` with dilated convolutions, yielding an output stride of 8. The main segmentation head (`FCNHead`) processes the 2048-channel features from `layer4` using a 3×3 convolution that reduces the channel dimension to 512, followed by batch normalization, ReLU activation, dropout with probability 0.1, and a 1×1 convolution that produces C class logits. The logits are then resized directly to the input image dimensions $H \times W$ using bilinear interpolation with `align_corners=False`. When enabled, an auxiliary classifier applies the same head structure to the 1024-channel features from `layer3`, also at output stride 8, using 256 intermediate channels. Its output can provide an additional supervision signal during training. The primary segmentation prediction is obtained from the main output (`out`), without fusion with the auxiliary output.

4.3 DeepLabV3 and atrous convolutions

DeepLabV3, developed by Chen et al. [10], addresses the resolution loss caused by consecutive pooling operations by replacing standard downsampling with atrous (dilated) convolutions and multi-scale pooling modules.

4.3.1 Atrous convolution formulation

Chen et al. [10] define atrous convolution in one dimension as $y[i] = \sum_k x[i+r\cdot k]\, w[k]$ (Eq. 1 in [10]), where the atrous rate r is the stride with which the input signal is sampled. Equivalently, the input is convolved with an upsampled filter obtained by inserting $r-1$ zeros between two consecutive filter values along each spatial dimension [10]. Applied along both spatial dimensions to a 2D feature map $x[i,j]$ with a $K \times K$ filter $w[p,q]$, where $p, q \in \{-M, \ldots, M\}$ and $M = \frac{K-1}{2}$ for odd K, the operation becomes:

$$y[i,j] = \sum_{p=-M}^{M} \sum_{q=-M}^{M} x[i + r \cdot p,\ j + r \cdot q] \cdot w[p,q]. \tag{4.3}$$

Standard convolution is the special case $r = 1$ [10]. Because $r-1$ zeros are inserted in each of the $K-1$ gaps between filter values, the effective filter size K' is:

$$K' = K + (K-1)(r-1). \tag{4.4}$$

Géron gives the one-dimensional case: the filter $[1, 2, 3]$ dilated with rate 4 becomes $[1, 0, 0, 0, 2, 0, 0, 0, 3]$, of length $3 + 2 \times 3 = 9$ [16, Chapter 12]. Likewise, a 3×3 filter with rate $r = 6$ covers a 13×13 region ($3 + 2 \times 5 = 13$) with 9 learnable parameters. Atrous convolution therefore enlarges the receptive field without extra parameters [16, Chapter 12] and without reducing the spatial resolution of the feature maps [10].

4.3.2 Atrous Spatial Pyramid Pooling (ASPP)

Objects and agricultural corridors appear at varying visual scales depending on camera mounting angle and distance from the vehicle. DeepLabV3 captures contextual information across multiple scales using the Atrous Spatial Pyramid Pooling (ASPP) module.

The ASPP module operates in parallel on the high-level feature map extracted by the backbone and consists of five branches:

1. One 1×1 standard convolution with 256 filters and batch normalization.
2. Three 3×3 atrous convolutions with 256 filters each, using dilation rates $r \in \{6, 12, 18\}$ for an output stride of 16 (or $r \in \{12, 24, 36\}$ for stride 8).
3. One Image-Level Pooling branch: to incorporate global image context, this branch computes global average pooling across the spatial dimensions of the input tensor, processes the resulting $1 \times 1 \times C_{\text{in}}$ vector with a 1×1 convolution (256 filters) and batch normalization, and upsamples the output bilinearly to the original ASPP spatial resolution.

Implementation note: The TorchVision `deeplabv3_resnet50` implementation [19] employs a dilated ResNet-50 backbone with output stride 8, achieved by removing downsampling in `layer3` and `layer4` and introducing atrous convolutions. The ASPP module correspondingly uses dilation rates $r \in \{12, 24, 36\}$ for the three parallel atrous branches.

The outputs of all five branches are concatenated along the channel dimension, resulting in a tensor with $5 \times 256 = 1280$ channels:

$$F_{\text{concat}} = [F_{1\times1},\ F_{\text{atrous},r_1},\ F_{\text{atrous},r_2},\ F_{\text{atrous},r_3},\ F_{\text{image_pool}}]\,. \tag{4.5}$$

This concatenated representation is projected back to 256 channels via a 1×1 convolution with batch normalization [10]; TorchVision follows it with ReLU activation and dropout ($p = 0.5$) [19]. The TorchVision implementation then applies a 3×3 convolution ($256 \to 256$ channels) with batch normalization and ReLU before the final 1×1 classification layer maps to C class logits. The output is upsampled via bilinear interpolation to image resolution $H \times W$. The ASPP parallel branch layout is depicted in Figure 4.2.

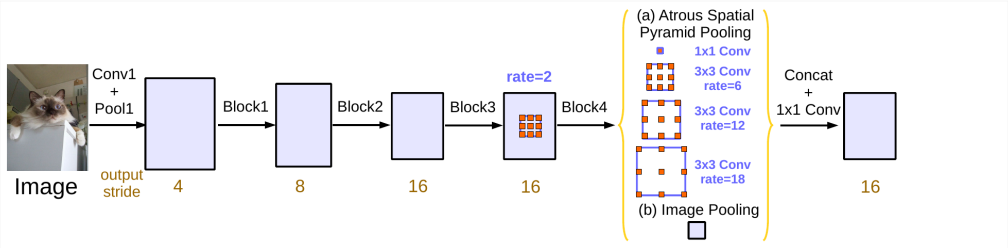


Figure 4.2: DeepLabV3 architecture featuring the Atrous Spatial Pyramid Pooling (ASPP) module with parallel atrous convolutions and image-level pooling (adapted from Chen et al. [10]).

4.4 MobileNetV3 and the Lite Reduced ASPP (LR-ASPP) head

DeepLabV3 with a ResNet-50 backbone provides high segmentation accuracy, but it is the slowest of the three networks in the offline comparison of Chapter 5 (86.90 ms mean recorded inference time). For mobile devices, Howard et al. [11] developed MobileNetV3, tuned to mobile phone CPUs, together with the Lite Reduced ASPP (LR-ASPP) segmentation head.

4.4.1 Depthwise separable convolutions

The building block that MobileNet architectures inherit from MobileNetV1 is the depthwise separable convolution [11], which factors a standard convolution into two separate steps: a spatial depthwise convolution and a channelwise pointwise convolution.

Consider an input feature map $X \in \mathbb{R}^{H \times W \times C_{\text{in}}}$ and an output feature map $Y \in \mathbb{R}^{H \times W \times C_{\text{out}}}$, processed with spatial filter size $D_K \times D_K$.

- **Standard convolution:** Each filter channel interacts with every input channel simultaneously. The computational cost, measured in Multiply-Accumulate (MAC) operations, is:

$$\text{Cost}_{\text{standard}} = H \cdot W \cdot C_{\text{in}} \cdot C_{\text{out}} \cdot D_K^2. \tag{4.6}$$

- **Depthwise convolution:** Applies a single $D_K \times D_K$ spatial filter per input channel independently, without channel cross-talk:

$$\text{Cost}_{\text{depthwise}} = H \cdot W \cdot C_{\text{in}} \cdot D_K^2. \tag{4.7}$$

- **Pointwise convolution:** Applies C_{out} filters of size 1×1 across all channels to compute linear combinations of the depthwise outputs:

$$\text{Cost}_{\text{pointwise}} = H \cdot W \cdot C_{\text{in}} \cdot C_{\text{out}}. \tag{4.8}$$

The total cost of a depthwise separable convolution is the sum of Eqs. (4.7) and (4.8):

$$\text{Cost}_{\text{sep}} = H \cdot W \cdot C_{\text{in}} \cdot D_K^2 + H \cdot W \cdot C_{\text{in}} \cdot C_{\text{out}}. \tag{4.9}$$

Comparing Eq. (4.9) to Eq. (4.6) yields the computational reduction ratio:

$$\frac{\text{Cost}_{\text{sep}}}{\text{Cost}_{\text{standard}}} = \frac{H \cdot W \cdot C_{\text{in}} \cdot D_K^2 + H \cdot W \cdot C_{\text{in}} \cdot C_{\text{out}}}{H \cdot W \cdot C_{\text{in}} \cdot C_{\text{out}} \cdot D_K^2} = \frac{1}{C_{\text{out}}} + \frac{1}{D_K^2}. \tag{4.10}$$

For 3×3 filters ($D_K = 3$), the ratio is $\frac{1}{64} + \frac{1}{9} \approx 0.13$ for $C_{\text{out}} = 64$ and approaches $\frac{1}{9} \approx 0.11$ as C_{out} grows. The parameter count follows the same ratio, since it equals the cost without the factor $H \cdot W$. Depthwise separable convolutions thus reduce arithmetic operations and parameters by about 87–89% relative to standard convolutions.

4.4.2 Inverted residual blocks with linear bottlenecks

In classical ResNet-50 bottleneck blocks, the input channels are first compressed via a 1×1 convolution, processed with a 3×3 convolution at reduced dimensionality, and

expanded back to the original high channel count (wide → narrow → wide). This structure reduces computational cost while maintaining representational capacity.

MobileNetV3 adopts the inverted residual block with linear bottleneck introduced by MobileNetV2, which reverses this design (narrow → wide → narrow) [11]:

1. **Expansion layer:** A 1×1 pointwise convolution expands the low-dimensional input C_{in} to a higher-dimensional space $e \cdot C_{\text{in}}$, where the expansion factor e varies between blocks (from 1 to 6 in MobileNetV3-Large [11, Table 1]).

2. **Depthwise layer:** A 3×3 or 5×5 depthwise convolution performs spatial filtering in the expanded channel space.

3. **Linear bottleneck:** A 1×1 projection layer compresses the channels back to C_{out}. No non-linear activation follows this projection: the narrow input and output of the block remain linear, and the non-linearities act only in the expanded space [11].

When the input and output have the same number of channels and the stride is 1, a residual shortcut connection adds the input directly to the block output, as illustrated in Figure 4.3a.

4.4.3 Squeeze-and-Excitation (SE) channel attention

To recalibrate channel responses, MobileNetV3 inserts Squeeze-and-Excitation (SE) modules in selected inverted residual blocks (8 of the 15 blocks of MobileNetV3-Large [11, Table 1]). The module is placed after the depthwise convolution, so that it acts on the expanded representation:

1. **Squeeze:** Global spatial average pooling aggregates the spatial dimensions of feature map $U \in \mathbb{R}^{H' \times W' \times C'}$ into a channel descriptor vector $z \in \mathbb{R}^{C'}$:

$$z_c = \frac{1}{H'W'} \sum_{i=1}^{H'} \sum_{j=1}^{W'} U_{i,j,c}. \tag{4.11}$$

2. **Excitation:** Two fully connected layers parameterized by weights $W_1 \in \mathbb{R}^{\frac{C'}{4} \times C'}$ and $W_2 \in \mathbb{R}^{C' \times \frac{C'}{4}}$ model non-linear channel relationships; the SE bottleneck is fixed to $1/4$ of the channels of the expansion layer [11]:

$$s = \sigma\left(W_2 \cdot \text{ReLU}(W_1 z)\right), \tag{4.12}$$

   where $\sigma(\cdot)$ denotes the hard-sigmoid activation function.

3. **Scale:** The input tensor U is rescaled channel-wise by the activation vector s:

$$\tilde{U}_{i,j,c} = s_c \cdot U_{i,j,c}, \qquad \forall c \in \{1, \ldots, C'\}. \tag{4.13}$$

4.4.4 Efficient non-linearities: hard-swish and hard-sigmoid

The swish non-linearity, defined as $\text{swish}(x) = x \cdot \sigma(x)$, improves the accuracy of neural networks when used as a drop-in replacement for ReLU. However, the sigmoid $\sigma(x) = (1 + e^{-x})^{-1}$ is much more expensive to compute on mobile devices and makes accuracy difficult to maintain in fixed-point arithmetic [11].

MobileNetV3 replaces standard sigmoid and swish with piece-wise linear hard approximations based on $\text{ReLU6}(x) = \min(\max(x, 0), 6)$:

$$\text{h-sigmoid}(x) = \frac{\text{ReLU6}(x+3)}{6}, \tag{4.14}$$

$$\text{h-swish}(x) = x \cdot \text{h-sigmoid}(x) = x \cdot \frac{\text{ReLU6}(x+3)}{6}. \tag{4.15}$$

Eqs. (4.14) and (4.15) contain no exponential. Howard et al. found no discernible difference in accuracy between the hard and smooth versions, while the piece-wise form reduces latency on mobile CPUs [11]. Because the cost of a non-linearity decreases as activation maps shrink, MobileNetV3 uses h-swish only in the second half of the network and ReLU in the earlier layers [11].

4.4.5 The Lite Reduced ASPP (LR-ASPP) architecture

The DeepLabV3 ASPP head applies four parallel convolutional branches with 256 channels each plus an image-pooling branch. Howard et al. proposed the Lite Reduced ASPP (LR-ASPP) head as a lighter alternative: it deploys global average pooling in a fashion similar to the Squeeze-and-Excitation module and uses only one 1×1 convolution in the pooling branch [11].

The LR-ASPP architecture uses features from two distinct stages of the MobileNetV3-Large backbone:

- A low-resolution, high-semantic feature map $F_{16} \in \mathbb{R}^{\frac{H}{16} \times \frac{W}{16} \times 960}$ extracted at output stride 16.
- A high-resolution, low-semantic feature map $F_8 \in \mathbb{R}^{\frac{H}{8} \times \frac{W}{8} \times 40}$ extracted at output stride 8.

The TorchVision LR-ASPP implementation (`lraspp_mobilenet_v3_large`) [20] processes the two feature streams as follows:

1. The stride-16 feature map F_{16} passes through two parallel branches:
   - **CBR path:** A 1×1 convolution followed by batch normalization and ReLU activation projects F_{16} to an intermediate representation $X \in \mathbb{R}^{\frac{H}{16} \times \frac{W}{16} \times C_{\text{inter}}}$, with $C_{\text{inter}} = 128$:

$$X = \text{ReLU}\left(\text{BN}\left(\text{Conv}_{1\times1}(F_{16})\right)\right). \tag{4.16}$$

- **Global context branch:** Applies global average pooling followed by a 1×1 convolution and a standard sigmoid activation to produce a channel-wise gating vector $G \in \mathbb{R}^{1\times1\times C_{\text{inter}}}$:

$$G = \sigma\left(\text{Conv}_{1\times1}\left(\text{GAP}(F_{16})\right)\right). \tag{4.17}$$

2. The CBR output X is modulated element-wise by the gating vector G and then upsampled by a factor of 2 via bilinear interpolation:

$$M_{16} = \text{BilinearUpsample}_{2\times}(X \odot G). \tag{4.18}$$

3. The upsampled context features M_{16} and the stride-8 features F_8 pass through **separate** 1×1 classification layers to produce class logits:

$$Z_{16} = \text{Conv}^{(16)}_{1\times1}(M_{16}), \qquad Z_8 = \text{Conv}^{(8)}_{1\times1}(F_8). \tag{4.19}$$

4. The two logit maps are **summed in logit space** to produce the final stride-8 prediction:

$$Z_{\text{fused}} = Z_{16} + Z_8. \tag{4.20}$$

5. A final $8\times$ bilinear upsampling step restores the prediction to the original image dimensions $H \times W$.

Implementation note: Unlike architectures that fuse feature representations before classification, LR-ASPP fuses predictions in logit space. This design allows each branch to learn independent classification boundaries while combining their complementary spatial resolutions. The global context gating uses a standard sigmoid activation (Eq. (4.17)), not the hard-sigmoid approximation employed in the MobileNetV3 inverted residual blocks.

By replacing the multi-branch parallel ASPP structure with a single global pooling gate and logit-space fusion, LR-ASPP reduces computational operations while retaining global scene context and boundary localization. The inverted residual backbone block and the complete LR-ASPP segmentation topology are illustrated in Figure 4.3.

4.5 Segment Anything Model 3 (SAM 3) for dataset annotation

Manually annotating agricultural images at pixel precision requires drawing polygons around irregular plant boundaries, inter-row soil, and covering material. The time required for this labeling motivated the use of SAM 3 to generate training

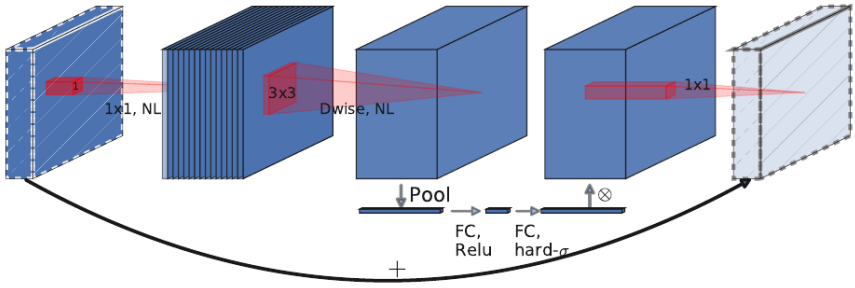


Figure 4. MobileNetV2 + Squeeze-and-Excite [20]. In contrast with [20] we apply the squeeze and excite in the residual layer. We use different nonlinearity depending on the layer, see section 5.2 for details.

(a) MobileNetV3 Inverted Residual block with Squeeze-and-Excitation channel attention.

V3-Small†	16.1	43	1.77	0.16

Table 6. Object detection results of SSDLite with different backbones on COCO test set. †: Channels in the blocks between $C4$ and $C5$ are reduced by a factor of 2.

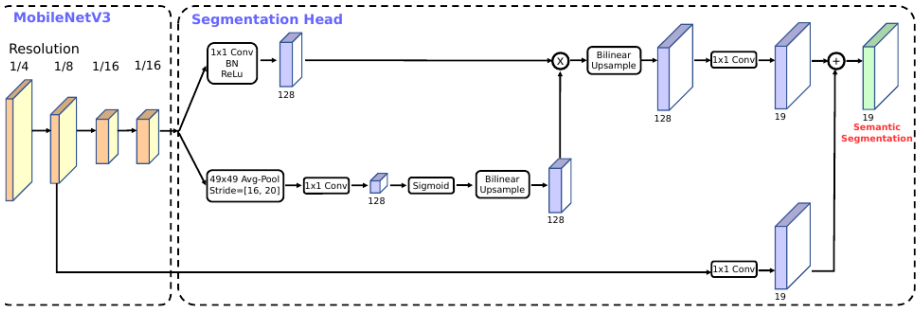


Figure 10. Building on MobileNetV3, the proposed segmentation head, Lite R-ASPP, delivers fast semantic segmentation results while mixing features from multiple resolutions.

(b) Lite Reduced ASPP (LR-ASPP) segmentation head mixing low- and high-resolution features.

Figure 4.3: MobileNetV3 and Lite Reduced ASPP (LR-ASPP) architecture: (a) inverted residual block combining depthwise separable convolution, linear bottleneck, and Squeeze-and-Excitation channel attention; (b) segmentation head combining stride-16 features modulated by global context gating with stride-8 fine features (adapted from Howard et al. [11]).

labels automatically (Chapter 5). Foundation models make it possible to generate candidate masks from prompts without training on the target crops.

Kirillov et al. [12] established the promptable segmentation paradigm with SAM, using Vision Transformers to generate binary segmentation masks from spatial prompts such as foreground/background points, bounding boxes, or coarse masks. Carion et al. [13] extended this paradigm in SAM 3 by introducing open-vocabulary concept segmentation driven by textual descriptions and visual exemplars.

4.5.1 Architecture overview

Unlike the original SAM architecture, which processes images through a single ViT encoder followed by a prompt-conditioned mask decoder, SAM 3 adopts a dual-architecture design comprising a **detector** for image-level segmentation and a **tracker** for video sequences. Both components share a unified **Perception Encoder (PE)** backbone that processes visual and linguistic inputs.

The SAM 3 detector follows a DETR-based paradigm with the following key components:

1. **Perception Encoder:** A shared vision-language backbone that encodes images into spatial feature representations. Unlike SAM's standalone ViT encoder, the PE is designed to handle both visual data and textual concept embeddings.

2. **Fusion Encoder:** Accepts unconditioned image embeddings from the PE and conditions them by cross-attending to prompt tokens derived from text descriptions (e.g., the prompts `"plants"`, `"central farm path"`, and `"plastic mulch film"` used in Chapter 5) or visual exemplars (bounding boxes around reference instances).

3. **Presence Token:** A learned global token, processed in the decoder together with the object queries, that decouples recognition from localization. It predicts only whether the prompted concept is present in the image, while each object query solves the localization problem; the final score of each query is the product of its own score and the presence score. Carion et al. report that this decomposition reduces the detector's sensitivity to false positives [13].

4. **Decoder:** Learned object queries cross-attend to the conditioned image embeddings from the fusion encoder, producing a set of binary segmentation masks and associated confidence scores for all instances of the target concept.

For video input, SAM 3 extends this architecture with a memory-augmented tracker that shares the frozen PE backbone with the detector. The tracker incorporates a memory encoder and memory bank that store object appearance across frames, enabling consistent temporal segmentation.

Figure 4.4 illustrates the original SAM architecture, which established the foundation for prompt-based segmentation; SAM 3 extends this paradigm with text-driven concept detection and video tracking capabilities.

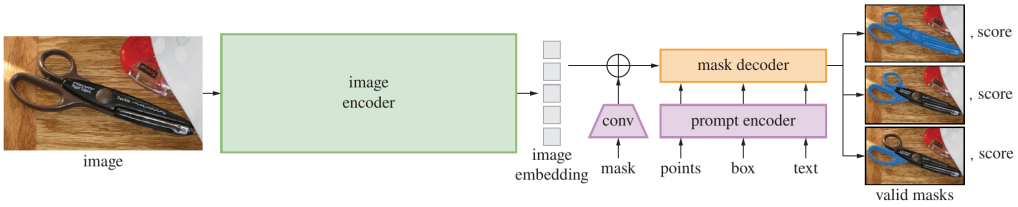


Figure 4.4: Original Segment Anything Model (SAM) architecture with ViT image encoder, prompt encoder, and mask decoder (adapted from Kirillov et al. [12]). SAM 3 extends this foundation with a dual detector-tracker architecture and open-vocabulary concept segmentation.

4.5.2 Operational distinction from onboard models

A key architectural distinction must be emphasized between the foundation model and the robotic segmentation network:

- SAM 3 has about 850 million parameters, of which about 450 million belong to the vision encoder and 300 million to the text encoder [13]. Its reported speed of 30 ms per image refers to an NVIDIA H200 data-center GPU [13], a hardware class not available on the robot, and the model was not evaluated on the onboard computer in this work.
- In this thesis, SAM 3 functions strictly as an **offline supervision generator**. Given text prompts representing crop and corridor concepts, SAM 3 generates dense candidate masks across lettuce, tomato, and cucumber image collections.
- The resulting masks are used without manual correction as training targets for LR-ASPP, the network selected for onboard perception after the comparison on the manually annotated lettuce dataset (Chapter 5). LR-ASPP recorded a mean offline inference time of 8.21 ms in that comparison.

4.6 Summary of architectural characteristics

Table 4.1 summarizes the characteristics of the three segmentation networks and of SAM 3 analyzed in this chapter, establishing the foundation for the dataset experiments and comparative evaluations presented in Chapter 5.

Table 4.1: Comparison of deep learning architectures used in this thesis. Recorded inference times are reported in Chapter 5.

Model	Backbone	Context mechanism	Target role	Primary characteristics
FCN	Dilated ResNet-50	Single-scale `FCNHead` on stride-8 features, auxiliary classifier during training	Benchmark segmentation	High parameter count, no multi-scale context module
DeepLabV3	Dilated ResNet-50	Multi-scale ASPP on stride-8 features	High-accuracy benchmark	Longest recorded inference time, multi-scale context
LR-ASPP	MobileNetV3-Large	Gated global pooling with logit-space fusion	Onboard segmentation for navigation	Compact architecture designed for mobile devices
SAM 3	Perception Encoder (vision-language)	DETR-based detector with presence token and text/visual prompts	Offline automatic annotation	Foundation model for open-vocabulary concept segmentation; not deployed onboard

Chapter 5

Dataset Preparation and Comparative Model Training

Following the theoretical analysis of deep learning architectures in Chapter 4, this chapter describes the datasets, annotation pipelines, and experimental model evaluations that guide the configuration for robotic integration.

The experimental study began by fine-tuning FCN, DeepLabV3, and LR-ASPP from pretrained weights on a manually annotated lettuce dataset for an empirical comparison. Because manual segmentation was time-consuming, this initial training was limited to lettuce rather than repeated across every target crop. Its outcome led to the adoption of LR-ASPP for embedded deployment.

The second experiment, carried out after the selection of LR-ASPP, tests whether networks trained on masks generated automatically by SAM 3, without any manual correction, can replace manual annotation. Manual masks were available for lettuce and were more accurate, but the aim was to evaluate labels that require no human work.

The experiment uses only lettuce images, because lettuce is the only crop with manual ground truth. SAM 3 generated three label sets for these images, with four, three, and two classes (Section 5.1.2), and a separate LR-ASPP network was trained on each. The predictions of each network are compared with the manual lettuce masks on the classes the two label sets share. The path class matters most, because it is the only one used for row following. The SAM 3 masks themselves are not compared with the manual masks.

The binary labeling was then extended to the complete image collection, labeled entirely by SAM 3, and a network trained on this collection is the one deployed on the robot. The results of this configuration are reported in Table 5.7.

This chapter details image acquisition, dataset partitioning, training parameters, and offline benchmark evaluations. Subsequent chapters detail geometric navigation

extraction and closed-loop robot validation.

5.1 Image acquisition and class definition

5.1.1 Image sources and acquisition conditions

[SPAZIO FIGURA / SCHEMA]

Sample field images across crops and corresponding training masks

Esempi visivi delle acquisizioni di campo per lattuga, pomodoro e cetriolo, con le corrispondenti maschere di annotazione impiegate per l'addestramento (ground truth manuale per la lattuga, maschere binarie del path autogenerate da SAM 3 per pomodoro e cetriolo).

Figure 5.1: Sample field images across crops and corresponding training masks

▶ **DA COMPLETARE: Origine delle immagini e condizioni di acquisizione**

Specificare la sorgente dei dati e le modalità di raccolta:

- Dispositivo di ripresa utilizzato (risoluzione e ottica), quota di acquisizione e angolazione.
- Condizioni operative nei campi (aziende agricole o serre sperimentali, condizioni meteo e fasce orarie).
- Numero complessivo di immagini raccolte per lattuga, pomodoro e cetriolo.

5.1.2 Semantic classes and labeling targets

The segmentation labels in this thesis use up to four semantic classes: background, plants, path (the traversable corridor between two crop rows), and covering material (plastic mulch film). The manual lettuce annotations use all four classes (Section 5.2.1).

The labels generated with SAM 3 (Section 5.2.2) follow three configurations, each used to train a separate network:

- **Four-class configuration:** background, plants, path, and covering material, matching the four classes of the manual lettuce annotations.

- **Three-class configuration:** background, plants, and path. No prompt targets the covering material.
- **Binary configuration:** background and path. This configuration is used both for the lettuce-only binary network and for the multi-crop dataset.

For row following, the path mask is the input of the geometric extraction in Chapter 6.

5.2 Dataset preparation

5.2.1 Lettuce dataset with manual annotation

The manually annotated lettuce dataset, identified as `Data`, contains 762 image–mask pairs distributed across eight folders. Its masks were prepared from the manual annotations stored in `annotations.json`. The same manual reference is used for the initial architecture comparison and for evaluating the models trained with SAM 3-generated labels. These manual masks are the ground truth for all evaluations in this chapter. They contain four classes (background, plants, path, and covering material), and the path class marks only the corridor the robot should follow, not other visible paths. The three lettuce autolabel datasets (`Data_insalataSAM_4class`, `Data_insalataSAM_3class`, and `Data_insalataSAM_2class`) contain images from the same eight lettuce folders, with masks generated by SAM 3 instead of the manual annotations.

5.2.2 Automated zero-shot annotation via SAM 3

Manual segmentation requires substantial human effort, which motivated testing whether SAM 3 can replace it. The experiment was carried out without any manual work: SAM 3 was run offline with text prompts, and its masks were used for training without inspection, filtering, or correction. Manual masks were available for lettuce and were more accurate, but keeping the unedited masks was an intentional choice, to measure how well a network performs when the annotation step is fully automatic.

All autolabel masks were generated using `SAM3SemanticPredictor` with the local `sam3.pt` checkpoint and a confidence threshold of 0.25. For each text prompt, SAM 3 returned zero or more candidate masks, and pixels with mask values greater than 0.5 were assigned to the corresponding class. When masks of different prompts overlapped, the class was assigned according to a fixed prompt order, listed below for each dataset: later prompts were written only on pixels still labeled as background, so earlier classes were never overwritten. Pixels not assigned by any prompt remained background. The prompts and their order are:

- `Data_insalataSAM_4class` (lettuce images): `plants` (class 1), then `plastic mulch film` (class 3), then `central farm path` (class 2)
- `Data_insalataSAM_3class` (lettuce images): `plants` (class 1), then `central farm path` (class 2)
- `Data_insalataSAM_2class` (lettuce images): `central farm path` only (class 1)
- `Data_allPlantsSAM2` (images of lettuce, tomato, and cucumber crops): `central path` only (class 1).

The SAM 3 masks are not compared directly with the manual annotations. Their quality enters the evaluation only through the networks trained on them (Section 5.4.1).



Figure 5.2: SAM 3 zero-shot autolabeling and mask composition workflow

5.2.3 Tomato and cucumber images

In addition to lettuce, the image collection includes tomato and cucumber crops. These images were annotated exclusively with SAM 3 and are included only in the multi-crop dataset (`Data_allPlantsSAM2`) used to train the binary network deployed on the robot. They were not included in the training sets of the lettuce-specific models.

Because no manual ground truth was created for tomato and cucumber, these crops are not part of the quantitative evaluations in Section 5.4. Similarly, the initial comparison among architectures (FCN, DeepLabV3, and LR-ASPP) was carried out exclusively on the lettuce dataset.

5.2.4 Dataset partitioning

The lettuce training runs use an 80% training, 10% validation, and 10% test partition. For the 762 image–mask pairs, this yields 609 training, 76 validation, and 77 test samples.

The training split is used to update the network weights. The validation split is used to compute the loss at the end of each epoch and to select the best checkpoint. The test split is not used during training and is used only afterwards, to compute the final scores. The three lettuce autolabel datasets contain the same 762 images as `Data`, in the same order, and the three autolabel projects use a split file identical to that of the manual model. The 77 test images are therefore the same physical images for all four lettuce networks, and none of them belongs to the 609 training images of any of these networks. The multi-crop binary network evaluated in Section 5.4 is trained on the complete image collection (`Data_allPlantsSAM2`, with images of lettuce, tomato, and cucumber crops). Before splitting this collection, the 77 lettuce test images were removed from it, so that none of them is used for training. The remaining 1882 of 1959 image–mask pairs were then split 80/10/10.

The split is random at the level of individual images: no test image is used for training, but each lettuce folder, which corresponds to one acquisition video recorded along a different path, contributes images to both the training and the test split. During training, the networks therefore see other photos taken along the same paths as the test images. The reported IoU values may overestimate performance on paths or fields not seen during training. This limitation applies to all networks of the comparison.

The realistic scenario uses only the test images from `insalata_grande1`, `insalata_grande2`, and `insalata_media1`: 42 of the 77 test images. These folders were recorded between parallel rows, in the area where the robot operates, from viewpoints the robot can reach. The other five folders were recorded from viewpoints the robot cannot reach and can show more than one path, while the manual masks label only the path the robot should follow. The realistic scenario is therefore a subset of the test split, not a separate test set, and its results describe only the robot's operating conditions.

5.3 Architecture comparison: FCN, DeepLabV3, and LR-ASPP

This section selects the segmentation architecture used in the rest of the thesis. FCN, DeepLabV3, and LR-ASPP were trained and compared on the manually annotated lettuce dataset.

5.3.1 Model selection and pretrained weights

FCN, DeepLabV3, and LR-ASPP were chosen from the implementations available in TorchVision. Their selection was practical, rather than the outcome of a comparison with every available architecture.

The implementation separates each network into a backbone, which extracts image features, and a segmentation head, which produces class scores at each output location. Transfer learning is implemented by loading pretrained parameters and adapting the output layers to the target labeling task, rather than initializing every network parameter from scratch.

For FCN and DeepLabV3, the selected implementations use a ResNet-50 backbone with pretrained segmentation weights. After these weights are loaded, the final convolution of the main classifier is replaced by a new 1×1 convolution with one output channel per target class (4 channels). The final convolution of the auxiliary classifier is replaced in the same way.

LR-ASPP is initialized in the same way. The implementation loads the default pretrained weights of the complete LR-ASPP model with a MobileNetV3-Large backbone. Its segmentation head ends with two 1×1 convolutions, one applied to the low-level features and one to the high-level features [20]. Both are replaced by new convolutions with 4 output channels. LR-ASPP has no auxiliary classifier.

The default weights of the three models were trained on a subset of COCO restricted to the 20 categories of Pascal VOC [18, 19, 20]. All three networks therefore start from pretrained segmentation weights of the same origin, and only their final output convolutions are newly initialized.

5.3.2 Training configuration and loss formulation

The optimization pipeline addresses class imbalance and follows a two-stage transfer learning schedule for all evaluated architectures.

Loss function and class weighting

Agricultural scenes exhibit a strong pixel imbalance between the navigable soil corridor, the plant canopy, and background areas. To prevent the optimization from biasing toward dominant categories, the models are trained with a weighted cross-entropy loss, denoted $\mathcal{L}_{\mathrm{WCE}}$, in which the contribution of each pixel is multiplied by the weight w_c of its ground-truth class c, defined in Eq. (5.1).

Class weights are computed exclusively on the training split $\mathcal{D}_{\mathrm{train}}$ to avoid information leakage from validation and test partitions. For each class c, the total pixel count $N_c = \sum_{k \in \mathcal{D}_{\mathrm{train}}} \sum_{i,j} \mathbb{I}(Y_{i,j}^{(k)} = c)$ is accumulated across all training masks. The balanced inverse frequency inv_c is then normalized by the mean of all inverse

frequencies:

$$\mathrm{inv}_c = \frac{\sum_{k=0}^{C-1} N_k}{C \cdot N_c}, \qquad w_c = \frac{\mathrm{inv}_c}{\frac{1}{C}\sum_{k=0}^{C-1} \mathrm{inv}_k}. \tag{5.1}$$

where C is the number of classes (four in this comparison), k indexes the training masks, $Y_{i,j}^{(k)}$ is the class of pixel (i, j) in mask k, and $\mathbb{I}(\cdot)$ equals 1 when its argument is true and 0 otherwise. A rare class therefore receives a large weight and a frequent class a small one. This normalization enforces $\frac{1}{C}\sum_{c=0}^{C-1} w_c = 1.0$.

For FCN and DeepLabV3, which incorporate an auxiliary classifier on the `layer3` features (output stride 8 in the dilated TorchVision backbone), the total training objective combines the primary and auxiliary losses with an auxiliary weighting factor of 0.4:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{WCE}}(\text{main}) + 0.4 \cdot \mathcal{L}_{\text{WCE}}(\text{aux}). \tag{5.2}$$

For LR-ASPP, which lacks an auxiliary head, training relies solely on the primary loss $\mathcal{L}_{\text{WCE}}(\text{main})$.

Data augmentation and two-stage optimization

Data augmentation is applied only to the training subset: random horizontal flips with probability 0.5, random rotations between $-10°$ and $10°$, and, with probability 0.8, a color jitter of brightness and contrast (factors in $[0.8, 1.2]$) and saturation (factors in $[0.85, 1.15]$) to handle variable outdoor illumination. The validation and test sets are not augmented. Images are not resized, and all images are normalized with the ImageNet mean and standard deviation.

The networks were trained on an NVIDIA GeForce RTX 2080 SUPER with 8 GB of memory, with a batch size of 2.

Optimization proceeds across 200 epochs in two distinct phases:

1. **Stage 1 (Warmup, Epochs 1–10):** Backbone parameters are frozen by setting `requires_grad = False`. Only the segmentation heads (and auxiliary classifiers when present) are optimized using AdamW with an initial learning rate of 10^{-3} and weight decay of 10^{-4}. A cosine-annealing scheduler with $T_{\max} = 200$ and $\eta_{\min} = 10^{-6}$ updates the learning rate after each epoch. In the first epochs, the newly initialized output layers make large errors, and the resulting large gradients could damage the pretrained weights. Freezing the reused layers during these epochs gives the new layers time to learn reasonable weights [16, Chapter 11].

2. **Stage 2 (Differential Fine-Tuning, Epochs 11–200):** At epoch 10, the backbone is unfrozen. A new AdamW optimizer is instantiated with decoupled parameter groups: the backbone is assigned a learning rate of 10^{-5} and the

segmentation heads a learning rate of 10^{-4}. The lower learning rate of the unfrozen backbone limits changes to its pretrained weights [16, Chapter 11]. The cosine-annealing scheduler is re-initialized across the remaining 190 epochs ($T_{\max} = 190, \eta_{\min} = 10^{-6}$).

Early stopping was not used: all three networks were trained for the full 200 epochs, so that the complete training and validation loss curves could be recorded. After each epoch, the model is evaluated on the unaugmented validation set. The weights are saved whenever the validation loss falls below its previous minimum, and the weights of the final epoch are saved separately.

5.3.3 Evaluation metrics and results

The three networks are evaluated on the 77 test images. The IoU is computed for each of the four classes, and the mIoU is their arithmetic mean, including background. The inference time is measured on the NVIDIA GeForce RTX 2080 SUPER used for training, one image at a time at its original resolution, and covers the forward pass and the argmax. The epoch column of Table 5.1 gives the epoch with the lowest validation loss, whose weights are evaluated.

FCN and DeepLabV3 reach the same mIoU (0.9253), 1.08 percentage points above LR-ASPP, and LR-ASPP has the lowest IoU in every class (Table 5.2). LR-ASPP is 7.2 times faster than FCN and 10.6 times faster than DeepLabV3.

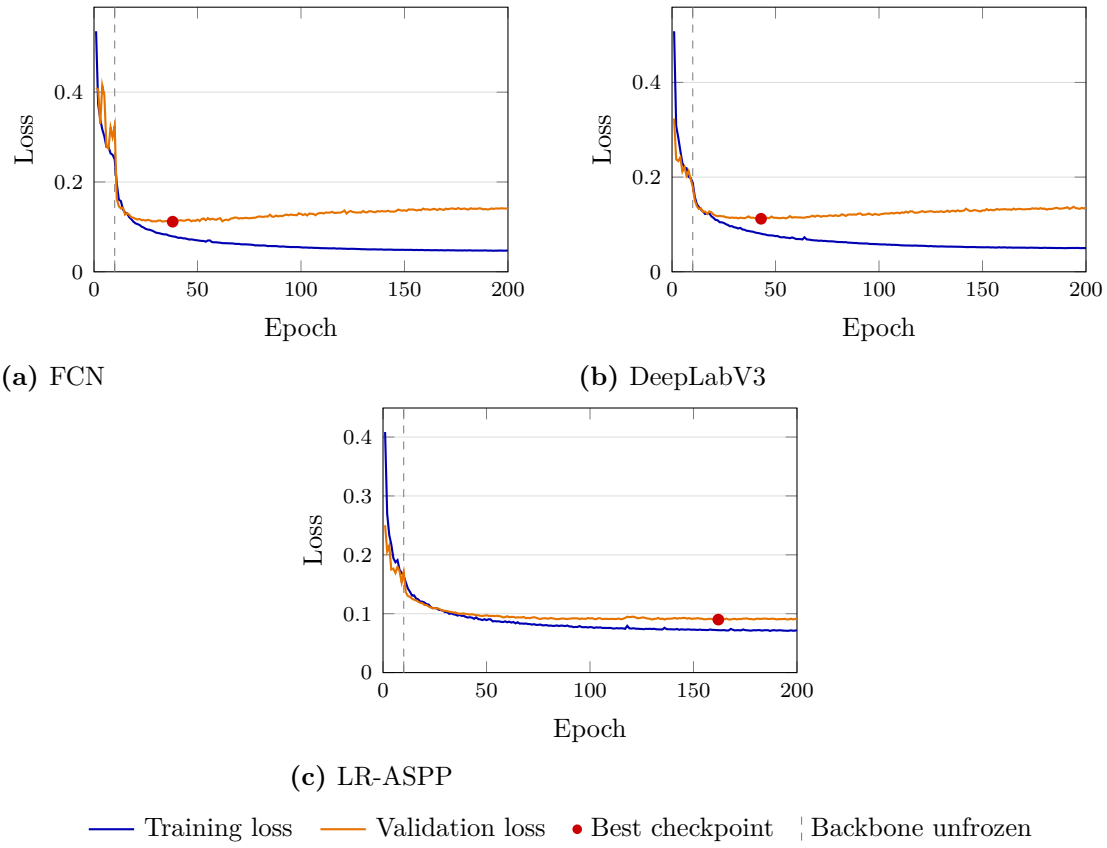
For FCN and DeepLabV3, the validation loss reaches its minimum early (Figure 5.3). After that, both networks start to overfit, and their validation loss rises while the training loss keeps decreasing. For LR-ASPP, the validation loss reaches a plateau around epoch 75 and does not rise until epoch 200. The losses of FCN and DeepLabV3 include the auxiliary term of Eq. (5.2), while those of LR-ASPP do not, so the absolute values are not comparable across networks.

Table 5.1: Architecture comparison on the 77 test images of the manual lettuce dataset. Parameter counts refer to the trained four-class models and include the auxiliary classifier of FCN and DeepLabV3, which is used only during training.

Model	Parameters (M)	Epoch	mIoU	Path IoU	Mean time (ms)	FPS
FCN	35.3	38	0.9253	0.9654	59.10	16.92
DeepLabV3	42.0	43	0.9253	0.9686	86.90	11.51
LR-ASPP	3.2	162	0.9145	0.9532	8.21	121.87

Table 5.2: Per-class IoU on the same 77 test images.

Model	Background	Plants	Path	Covering material
FCN	0.9493	0.9385	0.9654	0.8479
DeepLabV3	0.9494	0.9384	0.9686	0.8448
LR-ASPP	0.9439	0.9262	0.9532	0.8346



(a) FCN **(b)** DeepLabV3

(c) LR-ASPP

Figure 5.3: Training and validation loss per epoch of FCN, DeepLabV3, and LR-ASPP on the manual lettuce dataset. The best checkpoint is the epoch with the minimum validation loss. The backbone is unfrozen after epoch 10.

5.3.4 Selection of LR-ASPP

LR-ASPP was selected mainly for its speed. It was not the most accurate model, but its much shorter inference time offered the accuracy–latency compromise sought for robot perception. The subsequent SAM-supervised training runs therefore used LR-ASPP while varying the output label set.

These times were measured on the training PC, not on the robot. The timing of the complete perception and control pipeline on the robot is reported in Section 7.3.3.

5.4 Manual versus SAM 3 labels

5.4.1 Training configurations and evaluation

Table 5.3 summarizes the four LR-ASPP networks considered in this comparison. The manual baseline is trained on the manual labels. The other three networks are trained on SAM 3 labels and are called SAM 3-supervised networks below. This study is separate from the initial comparison of the three candidate architectures. All four networks are trained on lettuce images only, so that their predictions can be evaluated against the manual ground truth of the same crop. The extension of the binary configuration to the multi-crop dataset is described in Section 5.4.3.

Table 5.3: LR-ASPP training configurations in the supervision comparison. All networks are trained on the same lettuce images and evaluated against the manual lettuce ground truth. The epoch is that of the evaluated weights, those with the lowest validation loss.

Configuration	Labels	Output classes	Epoch
Manual baseline	Manual	Background, plants, path, covering material	162
SAM 3-supervised four-class	SAM 3	Background, plants, path, covering material	30
SAM 3-supervised three-class	SAM 3	Background, plants, path	38
SAM 3-supervised binary	SAM 3	Background, path	18

The SAM 3-supervised networks reach their minimum validation loss much earlier than the manual baseline, between epochs 18 and 38 against epoch 162. A possible explanation, not verified in this work, is the noise in the SAM 3 labels, which are also the labels of the validation split of these networks. Arpit et al. [21] observed that, when part of the training labels is random, a network first learns the patterns shared by the real data and only then fits the noisy labels, so its validation

accuracy reaches its maximum before the training accuracy becomes high. Their experiments concern image classification with randomly corrupted labels, while the SAM 3 label errors observed in this work are not random but depend on the prompts and on the scene, so their result supports this explanation only as an analogy.

Each network is evaluated with the IoU between its predictions and the manual lettuce labels of the test images:

$$\mathrm{IoU}_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FP}_c + \mathrm{FN}_c}, \tag{5.3}$$

where TP_c, FP_c, and FN_c are the true positive, false positive, and false negative pixels of class c. The mIoU is the arithmetic mean of the IoU of the evaluated classes.

Performance is evaluated on two subsets of the 77-image test partition:

- **Complete test set (77 images):** Spans all eight lettuce acquisition folders, including viewpoints that the robot cannot reach.
- **Realistic operational scenario (42 images):** Restricts evaluation to the test images of the three folders recorded between parallel rows, described in Section 5.2.4.

5.4.2 Results on the lettuce test images

All results are computed on the test split, both on the complete test set and in the realistic scenario. Each SAM 3-supervised network is compared with the manual baseline on the classes the two networks share. The background is not compared for the three-class and binary networks. In the three-class labels it also contains the covering material, and in the binary labels it also contains the plants and the covering material, so it does not match the background class of the manual masks. Table 5.4 compares the four-class networks on all classes, Table 5.5 compares the plants and path classes of the three-class network, and Table 5.6 compares the path class of the binary network.

Table 5.4: IoU of the four-class networks on the test set.

Scenario	Model	Background	Plants	Path	Covering material	mIoU
Complete test set	Manual baseline	0.9439	0.9262	0.9532	0.8346	0.9145
	SAM 3-supervised	0.4631	0.8081	0.4124	0.1719	0.4639
Realistic scenario	Manual baseline	0.9203	0.9485	0.9587	0.8178	0.9113
	SAM 3-supervised	0.3449	0.8757	0.8997	0.2495	0.5925

Table 5.5: IoU of the three-class network on plants and path. The mean is computed over these two classes only.

Scenario	Model	Plants IoU	Path IoU	Mean IoU
Complete test set	Manual baseline	0.9262	0.9532	0.9397
	SAM 3-supervised	0.8130	0.4124	0.6127
Realistic scenario	Manual baseline	0.9485	0.9587	0.9536
	SAM 3-supervised	0.8779	0.8966	0.8873

Table 5.6: Path IoU of the binary network trained on lettuce only.

Scenario	Model	Path IoU
Complete test set	Manual baseline	0.9532
	SAM 3-supervised	0.3917
Realistic scenario	Manual baseline	0.9587
	SAM 3-supervised	0.9013

In the four-class network, the IoU of the covering material and of the background is low. Visual inspection of the SAM 3 masks showed that SAM 3 segmented the covering material poorly in general and also assigned the greenhouse plastic cover to this class, which is background in the manual masks. This explanation comes from visual inspection only. With an mIoU of 0.5925 in the realistic scenario, against 0.9113 for the manual baseline, the four-class SAM 3 labels do not replace manual annotation when all four classes are needed.

All SAM 3-supervised networks lose much of their path accuracy on the complete test set (path IoU 0.39–0.41), while the manual model does not (0.9532). The drop comes from the five folders outside the operational scenario, whose images were taken from viewpoints the robot cannot reach. In these images the manual masks label only the path the robot should follow. Visual inspection showed that the SAM 3 path labels also cover other parts of the ground, such as adjacent paths and soil outside the corridor, especially where the path between the rows is not clearly visible. The networks learn this wider path class, and the IoU counts the additional ground pixels as false positives. The complete-test values therefore combine segmentation errors and this difference between the SAM 3 and manual labels. The two effects were not separated quantitatively.

In the realistic scenario, the path IoU of the three SAM 3-supervised networks is between 0.8966 and 0.9013, 5.7 to 6.2 percentage points below the manual baseline (0.9587). The three SAM 3-supervised networks differ from each other by less than 0.5 percentage points, so these data do not show that removing the plant and

covering material classes improves path segmentation. In any case, the geometric extraction uses only the path class (Chapter 6), so the binary configuration was chosen for the robot. Whether its accuracy is sufficient for row following is assessed in simulation and field tests (Chapter 8).

5.4.3 Extension to the multi-crop dataset and deployed network

After the lettuce study, the binary labeling was extended to the complete image collection, with images of lettuce, tomato, and cucumber crops, labeled entirely by SAM 3 with the `central path` prompt and without manual correction. The binary LR-ASPP network trained on this collection is the one deployed on the robot and used in the following chapters (Chapters 7 and 8).

The training data of the deployed network include the lettuce test images, so its IoU on them would not be an independent measure. A second network with the same configuration was therefore trained on the collection without these images (Table 5.7). Its results estimate the accuracy of the configuration, not of the exact deployed weights. On the complete test set it is clearly better than the lettuce-only binary network (0.6369 against 0.3917).

Table 5.7: Path IoU of the multi-crop binary network, trained without the lettuce test images.

Scenario	Model	Path IoU
Complete test set	Manual baseline	0.9532
	SAM 3-supervised binary, multi-crop	0.6369
Realistic scenario	Manual baseline	0.9587
	SAM 3-supervised binary, multi-crop	0.9071

Chapter 6

Geometric Extraction of the Local Navigation Reference

This chapter describes the path extractor used on the robot, implemented in `pf5.py` and called Method 4 in the experimental evaluation, and the three baseline extractors it is compared with. Method 4 extracts the path intervals in horizontal strips of the predicted mask and, starting from the bottom of the image, selects in each strip the interval that continues the corridor found below, keeping the chain of intervals that gives the most edge samples. It then fits the left and right edges of the corridor separately with iteratively reweighted least squares. The centerline is the mean of the two edge lines.

6.1 Edge and centerline models

The input is the label mask M predicted by the network, of height H and width W, with x increasing to the right and y downward. In the mask predicted by the binary network, the path pixels have value 1 and all other pixels have value 0. The two corridor edges are modeled as straight lines in the image,

$$x_L(y) = m_L y + q_L, \qquad x_R(y) = m_R y + q_R, \tag{6.1}$$

and the centerline is their mean,

$$x_C(y) = \frac{x_L(y) + x_R(y)}{2} = m_C y + q_C, \qquad m_C = \frac{m_L + m_R}{2}, \quad q_C = \frac{q_L + q_R}{2}. \tag{6.2}$$

Writing x as a function of y avoids infinite slopes for vertical lines. Method 4 estimates two separate lines, each with its own slope and intercept. The edges need not be parallel, so the modeled width can decrease toward the top of the

image because of perspective. The two lines can also represent a lateral offset or a heading error of the robot with respect to the row. All distances are in pixels of the input mask.

Each edge is fitted on its own samples with its own weights, so within the fit a sample far from one edge does not change the line of the other edge.

The processing stages are:

- Select a lower region of interest and clean the path mask (Section 6.2).
- Extract the path intervals in horizontal strips (Section 6.3).
- Associate intervals upward from each candidate seed and keep the best chain (Section 6.4).
- Fit the two edges separately (Section 6.5).
- Validate the edge geometry, assign a status from a confidence score, and return the centerline (Section 6.6).

6.2 Region of interest and mask cleaning

The region of interest (ROI) begins at $y_0 = \lfloor \rho H \rfloor$ and extends to the bottom of the image, with height $H_R = H - y_0$. The ratio ρ depends on the camera mounting and framing. It is set so that the ROI keeps only the part of the image that contains the path. On the robot, $\rho = 0.50$, so for a 224×224 mask the ROI covers rows 112 to 223.

The path pixels of M inside the ROI form a binary mask. It is cleaned with an opening (5×5 elliptical kernel) followed by a closing (9×9 rectangular kernel), and connected components with fewer than 500 pixels are removed. All larger components are kept. The resulting mask $\widetilde{M}$ (`path` in the code) is used for the interval extraction.

6.3 Horizontal strips and intervals

The ROI is divided into N strips of height $h = \lfloor H_R/N \rfloor$, indexed from the bottom. N is a parameter of the extractor, set to 16 on the robot. Strip s covers the ROI rows from $H_R - (s+1)h$ to $H_R - sh - 1$, and its height in the image is $y_0 + H_R - (s + 0.5)h$. On the robot, $h = 7$ px, so the strips cover all 112 rows of the ROI.

In each strip, a column is active if it contains at least one path pixel of $\widetilde{M}$. Runs of active columns separated by at most 30 inactive columns are merged into intervals $[\ell_i, r_i]$ of width $d_i = r_i - \ell_i + 1$. Intervals narrower than 8 px are discarded.

An endpoint within 2 px of the image border is flagged as clipped. An interval clipped on both sides is discarded when path pixels cover less than 70% of the strip.



Figure 6.1: Horizontal strip decomposition and column-wise projection for path interval detection

6.4 Bottom-up association and chain selection

The association starts from a seed interval and moves upward. In each strip, each candidate interval is compared with the last accepted interval $[\ell_p, r_p]$ of width d_p through the boundary displacement $\delta_i = \max(|\ell_i - \ell_p|, |r_i - r_p|)$. Among the candidates with $\delta_i \leq D_4$, the one with the lowest cost $J_{4,i}$ is accepted:

$$D_4 = \max(28,\, 0.5\, d_p), \qquad J_{4,i} = \delta_i + 0.4\, |d_i - d_p|. \tag{6.3}$$

A strip without a compatible candidate is skipped and the last accepted interval is kept as reference. The association stops at the fourth consecutive skipped strip (`max_gap_strips`=3). It is greedy and carries no information from one frame to the next.

Every interval of the bottom six strips is used as a seed, and each seed produces a chain of accepted intervals. A seed that already belongs to a chain found from a lower seed is skipped, because its chain would be the upper part of that chain. The selected chain is the one with the most endpoints not clipped at the image border, which are the samples available to the two edge fits (Section 6.5). Ties are broken by the lowest sum of the costs $J_{4,i}$. An interval clipped on both sides adds no samples, and an interval clipped on one side adds one sample, so these chains receive fewer samples than a chain of the same length along the corridor. If the selected chain has fewer than three intervals, the result is `lost`.

On the robot, the six-strip seed window covers 42 image rows and the three tolerated skipped strips cover 21 rows. Changing N or ρ changes h and therefore both extents.

6.5 Separate fitting of the corridor edges

The left edge is fitted on the left endpoints ℓ_i of the accepted intervals that are not clipped on the left, and the right edge on the right endpoints r_i that are not clipped on the right. Each edge needs at least two samples, otherwise the result is `lost`. There is no single-edge fallback. The two fits can use different strips.

Each fit starts with uniform weights and performs four reweighting iterations with $\sigma = 18$ px. At iteration t, the residuals of the current line give the new weights

$$e_i^{(t)} = x_i - m^{(t)} y_i - q^{(t)}, \qquad w_i^{(t+1)} = \frac{1}{1 + (e_i^{(t)}/\sigma)^2}, \tag{6.4}$$

and the line is refitted with `numpy.polyfit` of degree one. This function multiplies the unsquared residuals by the weights [22], so each iteration solves

$$(m, q) = \arg\min_{m,q} \sum_i \left[w_i (x_i - m y_i - q) \right]^2 , \tag{6.5}$$

and each squared residual is weighted by w_i^2. The weights are recomputed from the current line at every iteration, not accumulated, and the number of iterations is fixed.

[SPAZIO FIGURA / SCHEMA]

Reweighted line fitting of corridor boundaries and centerline construction

Diagramma del fit separato dei bordi sinistro (x_L) e destro (x_R) con pesi $1/(1 + (e/\sigma)^2)$ ricalcolati a ogni iterazione ($\sigma = 18$ px) e calcolo della mezzeria $x_C(y)$.

Figure 6.2: Reweighted line fitting of corridor boundaries and centerline construction

Before the final fit, the accepted intervals are checked for a crossing of the two edges. The check changes the result only when the modeled width falls below $d_{\min}$

between the lowest and the highest accepted interval. Let $\Delta m = m_R - m_L$ and $\Delta q = q_R - q_L$. If $\Delta m > 10^{-6}$, the edges converge upward and the modeled width reaches the minimum interval width $d_{\min} = 8$ px at $y_\times = (d_{\min} - \Delta q)/\Delta m$. When $y_\times$ lies below the highest accepted interval, the intervals above $y_\times$ are discarded and both edges are fitted again. This check is repeated at most three times, and it ends earlier when no crossing is found, when fewer than three intervals would remain, or when an edge has fewer than two samples. The remaining intervals are used in all the following steps.

The final residual of each sample is its absolute horizontal distance from its fitted edge. With $\bar{e}$ the mean over the samples of both edges, the fit score is

$$F = \mathrm{clip}(1 - \bar{e}/\sigma,0,1). \tag{6.6}$$

The reweighting reduces the effect of isolated outliers, but it cannot correct a wrong association that produces a consistent wrong line.

6.6 Validation, confidence, and output

The modeled corridor width is $D(y) = x_R(y) - x_L(y) = \Delta m\, y + \Delta q$, and rows with $D(y) \geq d_{\min}$ are valid. Let $y_{\text{top}}^{\text{obs}}$ and $y_{\text{bot}}^{\text{obs}}$ be the smallest and largest heights of the edge samples. If $|\Delta m| < 10^{-6}$, the edges are parallel, the check requires $\Delta q \geq d_{\min}$, and the valid range goes from $y_{\text{top}}^{\text{obs}}$ to $H - 1$. Otherwise, the width reaches $d_{\min}$ at $y_\times$ and the valid range is

$$y_v^{\text{top}} = \begin{cases} \max(y_{\text{top}}^{\text{obs}}, y_\times) & \Delta m > 0 \\ y_{\text{top}}^{\text{obs}} & \Delta m < 0 \end{cases}, \qquad y_v^{\text{bot}} = \begin{cases} H - 1 & \Delta m > 0 \\ \min(H - 1, y_\times) & \Delta m < 0 \end{cases}. \tag{6.7}$$

In this case the fit is rejected if the crossing lies inside the observed range, $y_{\text{top}}^{\text{obs}} + 0.05H < y_\times < y_{\text{bot}}^{\text{obs}}$, which indicates swapped edges or an association across different regions. It is also rejected if the valid range is shorter than $0.10(H - y_0)$, which is 11.2 px on the robot. A crossing outside this interval can only shorten the valid range.

The regularity of the corridor width is measured on the accepted intervals that are not clipped on either side, because the width of a clipped interval is not the corridor width. With $z_j = r_j - \ell_j$ for these K intervals in association order,

$$E_D = \frac{Q_{0.90}\left(\{|z_{j+2} - 2z_{j+1} + z_j|\}_{j=1}^{K-2}\right)}{\max(\mathrm{median}_j\, |z_j|,1)}, \qquad A = \mathrm{clip}(1 - E_D/0.30,0,1), \tag{6.8}$$

where $Q_{0.90}$ is the 90th percentile. If $K < 3$, $E_D = 0$. A width that changes linearly from one interval to the next gives zero second differences. Because the differences

are taken in sample order and not per pixel of height, skipped strips and excluded clipped intervals can also produce nonzero values.

The tracking score T is the number of accepted intervals divided by the number of strips whose center lies at or below y_v^{top}, limited to 1. G depends on the longest run $g_{\max}$ of strips skipped between two accepted intervals. The confidence is

$$G = \text{clip}(1 - g_{\max}/3,0,1), \qquad C_4 = 0.3125\,T + 0.3125\,F + 0.25\,A + 0.125\,G. \tag{6.9}$$

The result is `lost` if the geometric check fails, if $\bar{e} > 2\sigma$, or if $C_4 < 0.38$. Otherwise it is `degraded` for $C_4 < 0.60$ and `ok` for $C_4 \geq 0.60$. C_4 is a heuristic score, not a calibrated probability.

For a `lost` result, the output mask is empty and no centerline model is returned. Otherwise, the centerline is drawn from the first row of the highest accepted strip, or from y_v^{top} if this is lower in the image, down to y_v^{bot}. In each row its column is clipped to $[0, W-1]$, and the drawn line is dilated with a 3×3 elliptical kernel. The controller reads the lateral error from the analytic model $x_C(y)$, not from the drawn mask (Chapter 7).

Table 6.1 lists the parameters of Method 4 used on the robot.

6.7 Baseline extractors

Method 4 is compared with three baseline extractors. They have the same interface as Method 4, so switching extractor only changes the module imported by the controller.

6.7.1 Center points

The center-point extractor (`pf_centri.py`) uses the same mask cleaning and interval extraction as Method 4, but with $N = 24$ strips and a ROI that starts at $y_0 = \lfloor 0.30H \rfloor$. Each interval is reduced to its midpoint $c = (\ell + r)/2$ and its width $d = r - \ell + 1$. The seed is the interval of the bottom six strips with the most path pixels, with ties broken by width. Intervals are associated upward with the cost

$$J_{\text{center}} = |c^{(k+1)} - c^{(k)}| + 0.4\,|d^{(k+1)} - d^{(k)}|, \tag{6.10}$$

under the jump limit $|c^{(k+1)} - c^{(k)}| \leq \max(28, 0.5\,d^{(k)})$, with the same stopping rule and the same minimum of three intervals as Method 4. One line is fitted to the midpoints with four iterations of the weights of Eq. (6.4). Unlike Method 4, the initial weights are proportional to the pixel count of each interval (normalized to mean one), the weights multiply across iterations, and clipped intervals are not excluded. The score is $0.30T + 0.35F + 0.25A + 0.10G$, with T computed over all strips, A on the widths d of all accepted intervals, and G on the longest run of

Table 6.1: Parameters of Method 4 (`pf5.py`) used on the robot. Distances and areas are in pixels of the 224×224 mask.

Parameter	Value
Path class	1
ROI starting ratio ρ	0.50
Number of strips N	16
Opening / closing kernels	5×5 / 9×9
Minimum component area	500
Minimum interval width $d_{\min}$	8
Maximum merged gap	30
Image-border margin	2
Full-strip occupancy gate	0.70
Seed search window	6 strips
Jump limit D_4	$\max(28,\, 0.5\, d_p)$
Width-change cost coefficient	0.4
Consecutive skipped strips allowed	3
Residual scale σ / iterations	18 / 4
Edge-crossing truncation passes	3
Residual rejection threshold	$2\sigma = 36$
Crossing tolerance	$0.05H$
Minimum valid length	$0.10(H - y_0)$
Width-score scale	0.30
Lost / ok thresholds on C_4	0.38 / 0.60

skipped strips, including the run that ends the association. The result is `lost` if the mean residual exceeds 2σ or if the score is below 0.38, and `degraded` if it is below 0.60. The line is drawn from the top of the highest accepted strip to the bottom of the image. Because the midpoints are formed before the fit, a disturbance on one edge moves the midpoint and enters the centerline.

6.7.2 Column histogram

The column-histogram extractor (`pf_histogram.py`) is a simplified version of the column projection of SegMin. SegMin sums a plant mask, accumulated over recent frames and refined with depth, along the image columns, smooths the profile, and takes its minimum [3]. `pf_histogram` uses the path mask of a single frame, without depth or smoothing, and takes the maximum. Its ROI starts at $y_0 = \lfloor 0.30H \rfloor$, and the mask is cleaned with the same opening and closing as Method 4, keeping components of at least 300 pixels. The column counts of its cleaned ROI mask $\widetilde{M}_h$ are

$$p(x) = \sum_{y=0}^{H-y_0-1} \widetilde{M}_h(y, x), \qquad x \in [0, W-1]. \tag{6.11}$$

The target column x_{target} is the center of the widest run of columns whose count equals the maximum $p_{\max}$. The confidence is $p_{\max}/(H - y_0)$, with the same thresholds as Method 4 (0.38 and 0.60). The output is the vertical line $x = x_{\text{target}}$ from y_0 to the bottom of the image, so the method gives a lateral position but no heading.

6.7.3 RANSAC edges

RANSAC line fitting is used in agricultural row detection on points of the crop rows [1, 14]. The RANSAC extractor (`pf_ransac.py`) applies it to the edges of the segmented corridor, without strips or bottom-up association. Its ROI starts at $y_0 = \lfloor 0.30H \rfloor$ and the mask is cleaned as in Method 4. Among the components with at least 500 pixels, it keeps the one that maximizes its area plus ten times the top row of its bounding box, which favors large components that start lower in the image. In each ROI row of this component with width of at least 10 px, the leftmost and rightmost pixels are edge points, and at least ten rows are required.

Each edge is fitted with RANSAC [15]: in 100 trials, two points drawn by a generator with fixed seed (42) define a line, and the points at a horizontal distance of at most 18 px from it are inliers. The line with the most inliers is refitted by least squares on its inliers, and the centerline is the mean of the two edge lines, as in Eq. (6.2). The result is `lost` if the mean inlier residual exceeds 36 px, if the mean inlier ratio of the two edges is below 0.30, or if the score 0.35 (inlier ratio) + 0.30 (fit score) + 0.20 (vertical coverage) + 0.15 (width score) is below 0.38, and

`degraded` if this score is below 0.60. The fit score is $\text{clip}(1 - \bar{e}_{\text{in}}/18,0,1)$, with $\bar{e}_{\text{in}}$ the average of the mean inlier residuals of the two edges. The vertical coverage is the fraction of ROI rows with edge points. The width score is $\text{clip}(1 - E_w/0.30,0,1)$, with E_w the median absolute difference between the widths of consecutive edge rows divided by their median width. The line is drawn from the highest edge point to the bottom of the image. Because the seed is fixed, repeated runs on the same frame give the same result.

6.8 Structural differences and limitations

Table 6.2 summarizes how the four extractors differ in their code. It does not describe measured performance, which is reported in Chapter 8.

Table 6.2: Structural comparison of the four extractors.

Characteristic	`pf_histogram`	`pf_centri`	`pf_ransac`	Method 4
Output	Column x_{target}	Line	Line	Line
Heading estimate	No	Yes	Yes	Yes
ROI start ρ	0.30	0.30	0.30	0.50
Edges fitted separately	N/A	No	Yes	Yes
Bottom-up seed and jump limit	No	Yes	No	Yes

Method 4 has four structural limitations:

1. **Seed dependence.** If the bottom six strips contain no interval, for example because the mask in front of the robot is missing, the frame is rejected even if the upper part of the corridor is segmented. Within a chain the association is greedy, so at a fork the chain follows the branch with the lowest cost in the next strip, even if that branch ends soon after.

2. **Thresholds in pixels.** The jump limit, the merging gap, the minimum widths and areas, and σ are in pixels of the 224×224 mask, so they depend on the resolution and on the camera mounting. Camera pitch and roll on uneven ground move the corridor in the image, and a displacement above the jump limit stops the association early.

3. **Straight edges.** Both edges are straight lines, so curved rows and headland turns are not represented. Li et al. [6] use a cubic B-spline for curved paths.

4. **One-sided gaps.** Each accepted interval provides both edges. Where plants are missing on one side, the interval can widen or merge with a nearby region,

and if the displacement exceeds the jump limit for more than three consecutive strips, the association stops. If the missing side extends the interval to the image border, the endpoint on that side is clipped, and the frame is rejected when an edge has fewer than two non-clipped samples. The association stop and the clipped-edge rejection do not exist in `pf_histogram`, which tracks no edges.

Chapter 7

Robot Platform and Integration

This chapter describes the hardware and software components of the robot platform used for field validation. It covers the camera configuration, computing hardware, software architecture using ROS, and the deployment of the trained segmentation model for real-time inference.

7.1 Robot hardware and sensors

7.1.1 Robot platform

The experimental platform is a custom-built mobile robot prototype, named Demetra, developed for agricultural inspection and autonomous navigation in row-crop environments. The chassis features a four-wheel steering (4WS) and four-wheel drive (4WD) kinematic configuration. All four wheels are independently driven, and both the front and rear axles provide active steering.

Steering operates in a symmetric contra-steering mode (opposite-phase steering), where the front wheels steer by an angle $\delta_f = \delta$ and the rear wheels steer by an equal and opposite angle $\delta_r = -\delta$. This configuration constrains the instantaneous center of rotation (ICR) to lie on the transverse axis passing through the geometric center of the chassis ($x = 0$). Relative to a conventional front-wheel Ackermann steering model with wheelbase L, symmetric four-wheel contra-steering halves the turning radius from $R = L/\tan\delta$ to:

$$R = \frac{L}{2\tan\delta}. \tag{7.1}$$

This reduction in turning radius enables tight turns at row headlands without requiring the robot to swing wide into adjacent crop beds. In addition, the vehicle

sideslip angle at the center of geometry remains zero under ideal kinematic rolling conditions ($\beta = 0$), so the heading of the chassis aligns with the tangent to the path.

▶ DA COMPLETARE: Dimensioni fisiche e peso del telaio

Inserire i valori geometrici misurati sul prototipo reale:

- Passo L (distanza tra asse anteriore e posteriore in metri) e carreggiata T (distanza trasversale tra le ruote in metri).
- Raggio nominale della ruota r_w e altezza libera da terra del telaio (ground clearance).
- Massa totale del veicolo (kg) comprensiva di pacco batterie, sensori e payload.

The traction subsystem comprises four DJI RoboMaster M3508 brushless DC motors, each coupled to an integrated 19:1 planetary gearbox. Operating at a nominal voltage of 24 V, each motor unit delivers up to 3.0 N·m of continuous torque with a no-load output shaft speed of 482 rpm (peak torque up to 5.0 N·m). Speed regulation is handled by four individual DJI RoboMaster C620 electronic speed controllers (ESCs) implementing field-oriented control (FOC) over a CAN bus interface at 1 Mbps, with continuous current limits of 20 A per channel.

Active steering is actuated by four CubeMars AK70-10 KV100 robotic actuators. Each steering unit integrates a brushless DC motor, a 10:1 planetary gearhead, a 14-bit magnetic absolute encoder, and a motor driver in a single housing. A cross-roller bearing supports simultaneous axial, radial, and tilting loads transmitted from the ground through the wheel uprights. The actuators produce a continuous torque of 8.3 N·m and a peak torque of 24.8 N·m under 24 V operation. The integrated 14-bit encoder provides angular position feedback with a resolution of 0.022°, used for closed-loop position control over a secondary CAN bus network.

Low-level actuator control, safety monitoring, and sensor feedback are executed on a DJI RoboMaster Development Board Type C. The board houses an STM32F407IGH6 microcontroller (ARM Cortex-M4 core running at 168 MHz with hardware floating-point support). Two independent CAN bus controllers interface directly with the four traction ESCs and the four steering actuators. The board also incorporates a 6-axis inertial measurement unit (IMU) mounted on a mechanical vibration-damping structure. Deterministic firmware on the microcontroller executes position and velocity loops at fixed intervals, parses motion commands from the onboard computer, and transmits wheel encoder odometry and IMU telemetry.

7.1.2 Camera configuration

Visual sensing is provided by an Intel RealSense D435 camera mounted on the front section of the robot frame. The sensor connects to the onboard computer

[SPAZIO FIGURA / SCHEMA]

Physical mobile robot platform and front camera mounting setup

Fotografia della piattaforma mobile con indicazione della telecamera RGB frontale, dell'altezza di montaggio h dal terreno, dell'angolo di inclinazione (pitch) verso il suolo e dell'alloggiamento del calcolatore di bordo.

Figure 7.1: Physical mobile robot platform and front camera mounting setup

via a USB 3.2 Gen 1 interface. For the navigation pipeline, the camera operates in RGB streaming mode at a resolution of 640×480 pixels and a frame rate of 30 frames per second (fps). The stream is captured as JPEG-compressed frames to minimize transfer overhead across the ROS 2 transport layer. The camera is positioned at height h above the ground with a downward tilt angle α toward the inter-row path, providing a field of view that encompasses the corridor surface and adjacent crop boundaries in front of the vehicle.

7.1.3 Computing platform

Onboard processing is executed on an NVIDIA Jetson AGX Orin Developer Kit (64 GB version). The computing unit incorporates a 12-core ARM Cortex-A78AE v8.2 64-bit CPU operating at 2.20 GHz and an NVIDIA Ampere architecture GPU with 2048 CUDA cores and 64 Tensor cores. The module delivers up to 275 TOPS of sparse INT8 compute capability and 138 TFLOPS in FP16 Tensor Core operations.

The system shares 64 GB of unified LPDDR5 memory between the CPU and GPU over a 256-bit bus, providing a memory bandwidth of 204.8 GB/s. Persistent storage and dataset logging use a 512 GB NVMe SSD mounted over an M.2 PCIe Gen4 $\times 4$ interface at `/mnt/nvme`, leaving the 64 GB onboard eMMC memory dedicated to the base operating system.

The software stack runs on Ubuntu 22.04.5 LTS with Linux kernel `5.15.148-tegra` and NVIDIA Linux for Tegra (L4T) release R36.4.4 under JetPack 6.1. The complete robotics and deep learning environment is containerized using Docker and the NVIDIA Container Runtime, providing pass-through access to the GPU, hardware accelerators, and USB character devices. Within the container, ROS 2 Humble Hawksbill coordinates image acquisition, neural network inference with ONNX Runtime 1.23.0 using the `CUDAExecutionProvider`, geometric line extraction, and low-level serial communication.

7.2 Software architecture

7.2.1 ROS integration

The software pipeline is implemented in ROS 2. Figure 7.2 shows the node graph recorded on the robot during navigation. Three nodes are active. The RealSense node (`/camera/camera`) publishes the compressed color images. The controller node of the package `seg_controller_onnx` subscribes to these images and runs, in the same callback, the segmentation network, the path extractor, the error filter, and the steering law. It publishes the velocity commands on `/cmd_vel`. The serial driver node (`/demetra_node`) receives the commands and sends them to the

microcontroller. The controller node also publishes a text log on `/debug/text` and, when the debug video is enabled, three debug images on the other `/debug` topics. The debug video is used only during development and is disabled when the robot operates. The `/debug` topics are not part of the graph in the figure.

The image subscription keeps only the last message (`KEEP_LAST` with depth 1) with `BEST_EFFORT` reliability, so a frame that arrives while the callback is still running replaces the previous one instead of being queued.

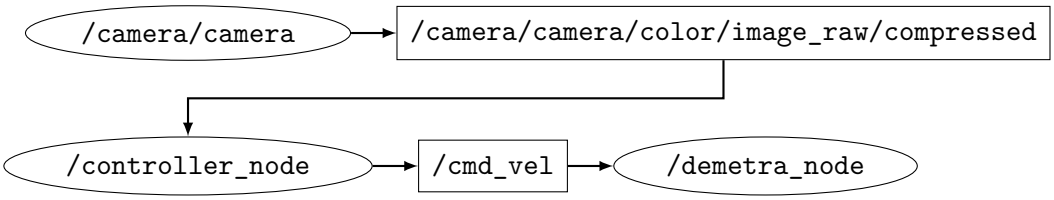


Figure 7.2: ROS 2 node graph on the robot, as shown by `rqt_graph`. Ellipses are nodes and rectangles are topics.

Perception node

The perception node receives camera images, runs the segmentation model, and publishes the path mask. Input images are preprocessed according to the training configuration before inference.

▶ **DA COMPLETARE: Dettagli implementativi del nodo di percezione**

Descrivere le operazioni eseguite alla ricezione di ogni frame:

- Decodifica dell'immagine compressa JPEG tramite `cv2.imdecode`.
- Ridimensionamento e normalizzazione del tensore d'ingresso (media e deviazione standard di ImageNet).
- Invocazione dell'inferenza tramite la sessione ONNX Runtime con `CUDAExecutionProvider`.
- Estrazione della maschera binaria di navigazione tramite operatore `argmax`.

Geometric extraction node

The geometric extraction node subscribes to the segmentation mask, applies one of the four extractors of Chapter 6 (the proposed Method 4, `pf4.py`, or one of the baselines `pf_centri`, `pf_histogram`, and `pf_ransac`), and publishes the navigation reference as a line or position message. All four extractors are tested on the robot (Chapter 8).

▶ **DA COMPLETARE: Integrazione dell'estratore geometrico e tracking temporale**

Dettagliare l'elaborazione geometrica:

- Suddivisione della maschera in strisce orizzontali e selezione dei candidati.
- Selezione dell'estrattore (`pf4.py`, `pf_centri.py`, `pf_histogram.py` o `pf_ransac.py`) e sua applicazione a ogni frame; indicare dove avviene il filtraggio temporale della linea (ad es. il filtro EMA descritto in questo capitolo).
- Gestione dei frame non validi (reiezione in caso di confidenza o supporto insufficiente).

Control node and error conditioning

The control node converts the image-space geometric reference into linear and angular velocity commands (v, ω) published on the `/cmd_vel` topic. Tracking errors extracted from the segmentation mask at reference row $y_{\text{ref}} = 0.50 \cdot H$ ($y = 112$ px for a 224×224 input resolution) are subject to sensor noise and transient occlusions. To ensure smooth closed-loop actuation without erratic steering adjustments, raw lateral pixel errors e_{px} pass through a multi-stage conditioning filter before entering the steering law.

Dual-regime recursive smoothing and jump rejection. The core filter operates as an exponential moving average (EMA) conditioned on the path finder tracking state $\text{status}_{\text{pf}} \in \{\text{ok}, \text{degraded}\}$. The filter smoothing factor α and jump threshold J_{thr} switch dynamically between two operating regimes:

$$(\alpha, J_{\text{thr}}) = \begin{cases} (0.20,\ 40.0 \text{ px}) & \text{if } \text{status}_{\text{pf}} = \text{ok}, \\ (0.18,\ 38.0 \text{ px}) & \text{if } \text{status}_{\text{pf}} = \text{degraded}. \end{cases} \tag{7.2}$$

At each sampled frame k, the discrepancy between incoming measurement $e_{\text{px},k}$ and prior filtered state e_{k-1} is evaluated as $\Delta_k = |e_{\text{px},k} - e_{k-1}|$. When $\Delta_k \leq J_{\text{thr}}$, the measurement is accepted as nominal and updated via standard EMA:

$$e_k = (1 - \alpha) e_{k-1} + \alpha\, e_{\text{px},k}, \tag{7.3}$$

with the consecutive jump counter reset to zero ($c_{\text{jump}} = 0$).

Conversely, when $\Delta_k > J_{\text{thr}}$, the counter increments ($c_{\text{jump}} \leftarrow c_{\text{jump}} + 1$). If the anomaly occurs on a single isolated frame ($c_{\text{jump}} = 1$), it is treated as a spurious segmentation outlier and heavily attenuated:

$$e_k = 0.80\, e_{k-1} + 0.20\, e_{\text{px},k}. \tag{7.4}$$

If the large displacement persists across consecutive frames ($c_{\text{jump}} \geq 2$), the filter classifies the variation as a genuine path change, resets $c_{\text{jump}} = 0$, and assigns $e_k = e_{\text{px},k}$ directly to avoid phase lag.

Temporal hold and command deadband. When the path finder reports a lost tracking state or when no valid centerline is detected ($e_{\text{px}} = \text{None}$), the filter initiates a zero-order temporal hold:

$$e_k = e_{\text{last_valid}} \quad \text{for } \Delta t \leq t_{\text{hold}}, \tag{7.5}$$

where $t_{\text{hold}} = 0.50$ s (≈ 15 frames at 30 fps). If perception dropouts exceed 0.50 s, the filter clears its internal state and issues an emergency stop command ($v = 0$, $\omega = 0$).

To eliminate high-frequency actuator jitter when the vehicle is aligned with the corridor, an output deadband of ± 4.0 px is applied:

$$e_{\text{cmd},k} = \begin{cases} 0.0 & \text{if } |e_k| < 4.0 \text{ px}, \\ e_k & \text{otherwise.} \end{cases} \tag{7.6}$$

The deadband alters only command output $e_{\text{cmd},k}$, preserving internal state e_k to avoid filter deadlock when crossing the zero-error threshold.

Velocity command generation. The filtered lateral error is normalized relative to the image half-width, $e_{\text{norm}} = e_{\text{cmd}}/(W/2)$, and scaled by the geometric extraction confidence $C \in [0, 1]$:

$$\omega = -K_{p,\text{lat}} \cdot (0.6 + 0.4 \cdot C) \cdot e_{\text{norm}}, \tag{7.7}$$

where $K_{p,\text{lat}} = 1.2$ is the lateral proportional gain. Calculated angular velocity is clamped within $[-\omega_{\max}, \omega_{\max}]$, while forward linear speed is set to $v = v_{\text{nom}}$. Table 7.1 summarizes the nominal parameters deployed on the robot platform.

7.2.2 Perception pipeline implementation

▶ DA COMPLETARE: Struttura a callback sincrona e carico delle immagini di debug

Descrivere il funzionamento del nodo unificato in un unico thread:

- Sequenzialità del ciclo: decodifica JPEG → inferenza ONNX → estrattore geometrico → generazione comandi → compressione e pubblicazione di 3 immagini di diagnostica visiva.
- Analisi dell'impatto prestazionale: la compressione JPEG delle 3 immagini di debug rappresenta una frazione significativa del tempo totale di callback.

Table 7.1: Nominal parameters of the error conditioning filter and steering controller deployed on Demetra.

Parameter	Symbol	Nominal value
Input resolution ($W \times H$)	–	224×224 px
Reference evaluation row	y_{ref}	112 px ($0.50 \cdot H$)
Smoothing factor (regime `ok`)	α_{ok}	0.20
Smoothing factor (regime `degraded`)	α_{deg}	0.18
Jump threshold (regime `ok`)	$J_{\text{thr,ok}}$	40.0 px
Jump threshold (regime `degraded`)	$J_{\text{thr,deg}}$	38.0 px
Outlier persistence window	N_{persist}	2 frames
Temporal hold duration	t_{hold}	0.50 s ($\approx$ 15 frames)
Output deadband	δ_{db}	$\pm$4.0 px
Lateral proportional gain	$K_{p,\text{lat}}$	1.2
Maximum angular velocity	ω_{max}	0.58 rad/s
Nominal linear speed	v_{nom}	0.20 m/s

7.2.3 Control interface

The control node translates the guidance reference into velocity commands published as `geometry_msgs/msg/Twist` messages on the `/cmd_vel` topic. These commands specify the forward linear speed v along the longitudinal chassis axis and the angular velocity ω about the vertical yaw axis.

A dedicated serial communication node, `demetra_serial_driver`, interfaces the ROS 2 environment with the low-level STM32 microcontroller. The physical connection uses an FTDI USB-to-UART TTL converter operating at a baud rate of 115200 bps with an update loop running at 100 Hz (10 ms period). Each transmitted packet consists of 24 bytes containing six 32-bit floating-point values followed by a checksum for transmission validation.

On the microcontroller, inverse kinematics converts the target (v, ω) command into individual steering angles and wheel rotation speeds according to the symmetric 4WS model. For a vehicle with wheelbase L and track width T, the nominal steering angle δ is:

$$\delta = \arctan\left(\frac{L\omega}{2v}\right), \tag{7.8}$$

with the front axle steering at $+\delta$ and the rear axle steering at $-\delta$. The individual turn radius for the center of each axle is $R = L/(2\tan\delta)$. To account for track width T and reduce wheel scuffing during cornering, individual steering angles for the left and right wheels follow dual-Ackermann geometry:

$$\delta_L = \arctan\left(\frac{L/2}{R - T/2}\right), \qquad \delta_R = \arctan\left(\frac{L/2}{R + T/2}\right), \tag{7.9}$$

with $\delta_{FL} = \delta_L$, $\delta_{FR} = \delta_R$, $\delta_{RL} = -\delta_L$, and $\delta_{RR} = -\delta_R$. The corresponding wheel linear velocities v_i for each wheel $i \in \{FL, FR, RL, RR\}$ are:

$$v_i = \omega\sqrt{\left(\frac{L}{2}\right)^2 + R_i^2}, \tag{7.10}$$

where R_i is the lateral distance from the ICR to the wheel center plane. Target wheel angular velocities $\omega_{w,i} = v_i/r_w$ (with wheel radius r_w) are converted into motor speed setpoints for the C620 ESCs via the 19:1 gear ratio, while the steering angles are tracked by the AK70-10 actuators via the internal 14-bit encoder position loops.

7.3 Deployment configuration

7.3.1 Model export to ONNX

The deployed LR-ASPP network (the multi-crop binary model `best_allsam2.pth`, Section 5.4.3) is exported from PyTorch to ONNX with `torch.onnx.export`.

The export script rebuilds the two-class LR-ASPP, loads the checkpoint weights, and wraps the network so that it returns only the main output tensor, that is, the per-pixel class logits; softmax and argmax are not part of the exported graph. The example input is a float32 tensor of size $1 \times 3 \times 224 \times 224$, with the batch dimension declared dynamic and the spatial size fixed at 224×224. The export uses opset 17 with constant folding, and the exported graph is validated with `onnx.checker.check_model`. The ONNX file used on the robot, `lraspp_224x224_allsam2.onnx`, was also compared with the PyTorch checkpoint on the same deterministic random input of size $1 \times 3 \times 224 \times 224$: the maximum absolute difference between the logits was 5.6×10^{-5} and the mean absolute difference 4.9×10^{-6}, so the two outputs agree within a relative tolerance of 10^{-3} and an absolute tolerance of 10^{-4}. This comparison was run with PyTorch 2.6.0 and ONNX Runtime 1.23.2 on the CPU.

On the robot, camera frames are resized to 224×224 before inference, to match this input size. The inference times in Table 5.1 were instead measured at the original image resolution, so they do not describe the onboard model.

7.3.2 Real-time inference

The deployed model processes camera frames at the rate required for closed-loop control. Input preprocessing matches the training transformations. The segmentation mask is decoded and passed to the geometric extraction stage.

▶ DA COMPLETARE: Configurazione del runtime ONNX

Specificare i dettagli dell'ambiente di esecuzione:

- Configurazione dell'allocazione di memoria e provider attivo (`CUDAExecutionProvider`).
- Tipologia di quantizzazione/precisione (es. FP32 standard vs FP16) e consumo di memoria VRAM sulla GPU Jetson.
- Specifiche del tensore d'ingresso (normalizzazione in $[0,1]$ o con media e deviazione standard di ImageNet, risoluzione) e del tensore di uscita (logit o indici di classe, risoluzione).

7.3.3 Timing and latency

The processing times were measured on the Jetson AGX Orin by passing 100 compressed camera frames, recorded in the `insalata_grande1` folder, through the callback of the controller node, after 5 warm-up frames. For each frame, the node records the inference time, the time of the path extractor (`pf4.py`, an earlier version of Method 4 that selected the seed by edge support and used 24 strips), and the total time of the callback. The first 50 frames were processed with the

debug video enabled, which renders and compresses the debug images, and the other 50 with the debug video disabled. Table 7.2 reports the mean and standard deviation of each time.

Table 7.2: Processing times of the deployed pipeline on the NVIDIA Jetson AGX Orin, with `pf4.py` as path extractor (mean ± standard deviation over 50 frames per condition).

Time (ms)	With debug video	Without debug video
Inference (ONNX Runtime)	9.92 ± 0.07	9.76 ± 0.07
Path extractor	8.88 ± 0.20	8.64 ± 0.18
Total callback time	53.14 ± 0.97	23.09 ± 0.21

Inference and path extraction take about 19 ms in both conditions. The rest of the total time belongs to the other operations of the callback, which were not measured separately: 4.70 ms without the debug video and 34.35 ms with it, so the debug video adds about 30 ms per frame. With the debug video enabled, the total time of 53.14 ms limits the processing rate to 18.8 frames per second, below the 30 fps of the camera. Without the debug video, the total time of 23.09 ms is shorter than the 33.3 ms frame period of the camera, so the processing time no longer limits the rate at which frames are processed. When the robot operates the debug video is disabled, so the second column of Table 7.2 describes the robot in operation.

7.3.4 Software-in-the-Loop pre-validation

Before deploying the navigation pipeline on the physical robot, the stability of the full closed-loop chain (segmentation, geometric extraction, error conditioning, steering law, and vehicle kinematics) was verified in a lightweight Software-in-the-Loop (SITL) environment running on a desktop workstation.

SITL architecture

The SITL node loads a single real RGB frame acquired by the Intel RealSense D435 camera on an actual crop row. Rather than rendering a synthetic 3D scene, the simulator models the robot's lateral displacement and heading variation as a horizontal translation of the camera viewport over the original image. The frame is padded with reflective borders (400 pixels per side) to allow displacements of up to ±0.45 m without introducing black edges.

The pixel shift Δx applied at each simulation step combines two terms derived

from the D435 optical parameters:

$$\Delta x = -y \cdot \frac{f_x}{D} + \theta \cdot f_x, \tag{7.11}$$

where y is the lateral offset of the robot from the corridor center (in meters), θ is the heading angle relative to the row axis (in radians), $f_x = 465$ px is the horizontal focal length of the D435 at 640×480 resolution, and $D = 1.25$ m is the lookahead distance corresponding to the reference evaluation row ($y_{\text{ref}} = 0.50 \cdot H$).

Kinematic model

The SITL integrates the same 4WS symmetric contra-steering kinematics implemented in the onboard firmware (`kinematics.cpp`), with identical parameters: wheelbase $L = 0.40$ m, wheel diameter $d_w = 0.192$ m, and mechanical steering saturation at $\pm 30.0°$. At each time step $\Delta t = 1/30$ s, the node receives (v, ω) from the `/cmd_vel` topic published by the real `controller_node`, computes the effective steering angle:

$$\delta = \arctan\!\left(\frac{L \cdot \omega}{2\,v}\right), \qquad \omega_{\text{eff}} = \frac{2\,v\,\tan\delta}{L}, \tag{7.12}$$

and updates the planar state:

$$\theta_{k+1} = \theta_k + \omega_{\text{eff}} \cdot \Delta t, \tag{7.13}$$

$$y_{k+1} = y_k - v \sin\theta_k \cdot \Delta t. \tag{7.14}$$

The translated frame is then published on the same ROS 2 topic used by the physical camera (`/camera/camera/color/image_raw/compressed`), so the `controller_node` processes it without any code modification.

Interactive perturbation testing

Keyboard inputs allow the operator to inject controlled lateral perturbations (± 4 cm or ± 10 cm) during execution, simulating sudden displacement events such as wheel slip on soft soil or an external push. This procedure was used to verify that the error conditioning filter and steering controller recover to the corridor center within approximately 10–15 frames (~ 0.5 s at 30 fps), without residual oscillations and without saturating the $\pm 30°$ mechanical steering limit. Table 7.1 lists the filter and control parameters validated through these SITL tests before field deployment.

Chapter 8

Experimental Validation

This chapter reports the experimental evaluation of the segmentation models, geometric extractors, and integrated navigation system. It covers offline evaluation using recorded images and, if completed, closed-loop field trials.

8.1 Offline evaluation protocol

8.1.1 Segmentation metrics

Segmentation performance is measured using intersection over union (IoU) for each semantic class. For a class c, IoU is defined as:

$$\mathrm{IoU}_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FP}_c + \mathrm{FN}_c}, \tag{8.1}$$

where true positives, false positives, and false negatives are computed against the manual reference masks.

Mean IoU (mIoU) averages the per-class IoU values. For the four-class configuration, mIoU includes background, plants, path, and covering material. Path IoU is the task-specific metric most relevant for navigation.

Architecture comparison results

The architecture comparison in Section 5.3.3 reports, on the 77 test images of the manual lettuce dataset, a path IoU of 0.9532 for LR-ASPP, against 0.9654 for FCN and 0.9686 for DeepLabV3. The recorded GPU inference times are 8.21 ms, 59.10 ms, and 86.90 ms. LR-ASPP accepts an mIoU about 1.1 percentage points lower in exchange for 7.2 to 10.6 times shorter recorded times.



Figure 8.1: Visual comparison of segmentation outputs on lettuce test images and on tomato and cucumber images

Supervision strategy results

The supervision comparison is reported in Section 5.4.2. On the test split, the manual model reaches a path IoU of 0.9532 on the complete test set and 0.9587 in the realistic scenario. The three networks trained only on SAM 3 lettuce labels reach path IoU values between 0.8966 and 0.9013 in the realistic scenario and between 0.3917 and 0.4124 on the complete test set; the 4-class network reaches an mIoU of 0.5925 in the realistic scenario. A network with the deployed multi-crop configuration, trained on the complete SAM 3 collection without the lettuce test images, reaches 0.9071 in the realistic scenario and 0.6369 on the complete test set; the deployed weights themselves have no independent IoU (Section 5.4.3).

8.1.2 Geometric reference accuracy

Geometric reference quality is assessed by evaluating four extraction strategies on identical segmentation masks: the proposed dual-edge extractor with continuity tracking (Method 4), the RANSAC-based contour baseline (`pf_ransac.py`), the center-point strip tracking baseline (`pf_centri.py`), and the column-histogram baseline (`pf_histogram.py`). Performance metrics comprise:

- Reference line parameters: estimated slope m_C and lateral intercept q_C (for linear models);
- Lateral deviation e_y and heading orientation error e_θ evaluated against manual centerline annotations;
- Sensitivity to asymmetric disturbances, quantified by the lateral deviation induced by unilateral crop intrusions or shadows;
- Rejection rate: percentage of test images where extraction fails validation checks (status classified as lost);
- Execution time per frame measured on the onboard NVIDIA Jetson AGX Orin platform.

Quantitative comparison of geometric extractors

Table 8.1 reports offline performance metrics for the four geometric extraction algorithms evaluated on recorded test sequences. Each algorithm processes identical segmentation masks predicted by the deployed binary network (`best_allsam2.pth`, Section 5.4.3) on the lettuce, tomato, and cucumber datasets.

Angular jitter σ_ω measures the standard deviation of the estimated centerline slope across 100 repeated executions on the same input frame. Deterministic

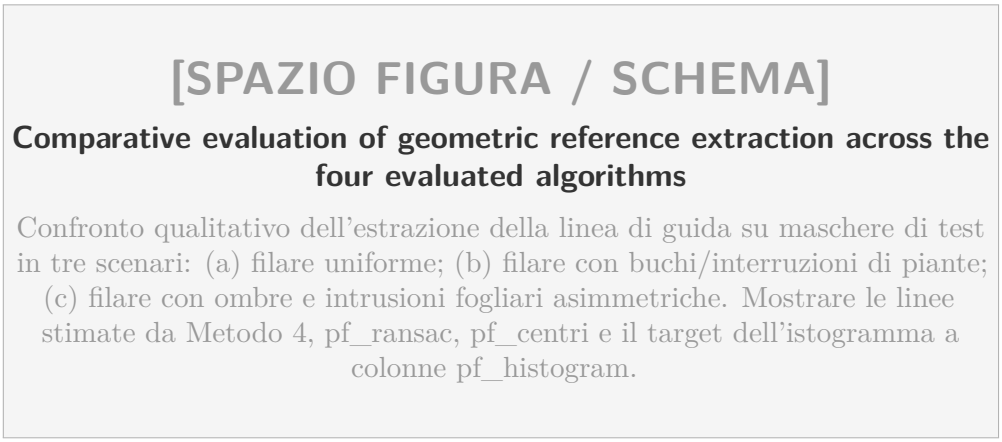


Figure 8.2: Comparative evaluation of geometric reference extraction across the four evaluated algorithms

Table 8.1: Offline benchmark of geometric extraction algorithms on multi-crop test sequences.

Metric	`pf_histogram`	`pf_centri`	`pf_ransac`	Method 4
Lateral RMSE (cm)	—	—	—	—
Angular RMSE (deg)	—	—	—	—
Angular jitter σ_ω (deg)	0.00	0.00	—	0.00
Rejection rate (%)	—	—	—	—
Execution time (ms)	—	—	—	—
Deterministic	Yes	Yes	Yes (fixed seed)	Yes

algorithms produce $\sigma_\omega = 0.00$ by definition. `pf_ransac` draws its samples from a generator with a fixed default seed, so it is also reproducible on the same frame; it shows non-zero jitter only if the seed changes between executions, which must be stated with its result.

Asymmetric disturbance sensitivity

Asymmetric corridor disturbances—such as protruding leaves, localized weeds, or shadows extending along one side—test the ability of each algorithm to maintain corridor centering when the segmentation mask exhibits unilateral bias. Algorithms that collapse bilateral boundaries into a single centerline estimate before fitting (such as `pf_centri`) shift laterally toward the intrusion. Method 4 and `pf_ransac` isolate each boundary and compute the centerline analytically, reducing sensitivity to unilateral noise.

Failure mode analysis

Extraction failures occur when validation checks reject the computed reference as unreliable. Common failure modes include:

- **Tracking loss:** Insufficient mask continuity across horizontal strips prevents bottom-up tracking initialization.
- **Angular drift from corrupted centroids:** Single outlier intervals dominate the centroid-based fit in `pf_centri`, producing reference lines with excessive slope.
- **Histogram target jumps:** The column-histogram minimum in `pf_histogram` shifts abruptly between frames when corridor width varies or when noise perturbs column sums.
- **RANSAC convergence variability:** In sparse or highly noisy masks, `pf_ransac` may fail to identify a consistent inlier set within the iteration budget, leading to sporadic rejections.

8.2 Field validation

Field validation evaluates the integrated navigation system on the physical robot Demetra operating autonomously inside crop rows. Unlike offline evaluation, which isolates the geometric extractor on recorded images, closed-loop field tests exercise the full chain from camera acquisition through segmentation, geometric fitting, error conditioning, and steering actuation in the presence of real-world disturbances.

8.2.1 Ground-truth reference: laser sensor

To assess the robot's physical centering independently of the onboard vision pipeline under test, closed-loop field validation integrates an auxiliary laser sensor.

▶ DA COMPLETARE: Dettagli del sensore laser utilizzato per i test in campo

Inserire le specifiche esatte del sensore laser impiegato nei test:

- Modello, costruttore e tipologia di misura (ad es. sensore di distanza laser, barriera ottica o profilometro).
- Configurazione di montaggio sul robot Demetra (posizione e orientamento).
- Interfaccia di comunicazione e frequenza di campionamento.

The laser measurement provides an independent reference signal to monitor the robot's lateral alignment along the crop row and evaluate whether the trajectory remains within the allowable tolerance corridor.

8.2.2 Field evaluation metrics

The following metrics are computed for each test run. All algorithms under comparison (Method 4, `pf_centri`, `pf_ransac`, and `pf_histogram`) are evaluated on the same physical row under equivalent conditions.

Laser centering rate

The laser centering rate P_{laser} quantifies the fraction of the autonomous run during which the robot remains centered within the laser reference corridor:

$$P_{\text{laser}} = \frac{T_{\text{on}}}{T_{\text{total}}} \times 100\ [\%], \tag{8.2}$$

where T_{on} is the cumulative time with positive laser detection and T_{total} is the total duration of the autonomous run. A higher P_{laser} indicates that the guidance algorithm maintains the robot closer to the physical corridor center for a larger fraction of the mission.

Number of centering losses

The number of centering loss events N_{exit} counts how many times the laser detector transitions from active (1) to inactive (0) during a run. Each event corresponds to the robot drifting outside the acceptable centering band. Frequent transitions indicate oscillatory tracking behavior, even if the overall centering rate P_{laser} remains acceptable.

Mission completion rate

The mission completion rate S_{mission} measures the fraction of the test row traversed autonomously without requiring manual intervention or emergency stop:

$$S_{\text{mission}} = \frac{L_{\text{auto}}}{L_{\text{row}}} \times 100\ [\%], \tag{8.3}$$

where L_{row} is the total length of the test row and L_{auto} is the distance covered before any intervention. A run terminated by collision with crop plants or by operator emergency stop records $S_{\text{mission}} < 100\%$.

Steering stability

Steering stability is evaluated from the angular velocity commands ω_k recorded on the `/cmd_vel` topic. Two indicators are computed:

- The standard deviation of angular velocity, $\sigma_\omega = \text{std}(\omega_k)$, which quantifies overall steering activity;
- The mean absolute jitter, $J_\omega = \frac{1}{N-1} \sum_{k=1}^{N-1} |\omega_{k+1} - \omega_k|$, which isolates high-frequency direction reversals independent of steady-state turning.

Lower values of both indicators correspond to smoother robot motion and reduced mechanical wear on the CubeMars AK70-10 steering actuators.

Effective forward speed

The effective forward speed v_{eff} is the ratio between the distance traversed and the elapsed time. If the error conditioning filter enters the temporal hold state and issues stop commands ($v = 0$), the effective speed drops below the nominal setpoint $v_{\text{nom}} = 0.20$ m/s. This metric captures how frequently each algorithm loses the guidance reference and forces the robot to halt.

8.2.3 Test conditions

▶ DA COMPLETARE: Descrizione delle condizioni sperimentali in campo

Dettagliare lo scenario delle prove sperimentali:

- Località del test, tipologia di coltura, sesto d'impianto (distanza tra i filari e altezza delle piante).
- Lunghezza del percorso navigato, tipologia del fondo (terreno lavorato, teli plastici pacciamanti, presenza di malerbe).
- Configurazione del sensore laser di riferimento (modello, posizionamento sul robot o nel filare, principio di misura).
- Condizioni meteorologiche e di luce (sole diretto, cielo coperto, ombre proiettate dalla vegetazione) e numero di ripetizioni effettuate per ciascun algoritmo.

8.2.4 Closed-loop navigation results

[SPAZIO FIGURA / SCHEMA]

Lateral tracking error and heading error profiles during inter-row autonomous navigation

Grafico dell'errore di inseguimento laterale e_y (cm) e dell'errore di orientamento e_θ (gradi) registrati nel tempo durante la navigazione del robot mobile all'interno dei filari coltivati.

Figure 8.3: Lateral tracking error and heading error profiles during inter-row autonomous navigation

▶ DA COMPLETARE: Risultati quantitativi di navigazione ad anello chiuso

Riportare i dati raccolti durante le prove in campo con il robot Demetra:

- Tabella comparativa tra i 4 algoritmi (Metodo 4, pf_ransac, pf_centri, pf_histogram) con le metriche definite in Sezione 8.2.2: P_{laser}, N_{exit}, S_{mission}, σ_ω, J_ω e v_{eff}.
- Grafico temporale del segnale del sensore laser sovrapposto alla velocità angolare comandata $\omega(t)$ per ciascun algoritmo.
- Analisi degli eventi critici: corrispondenza tra le anomalie del segnale laser e i frame classificati come `lost` dal path finder.

8.2.5 Qualitative observations

▶ DA COMPLETARE: Osservazioni qualitative sul comportamento del sistema

Discutere gli aspetti qualitativi emersi durante i test:

- Differenze visibili nella fluidità di guida tra Metodo 4 e pf_centri (oscillazioni laterali, scatti di sterzo).
- Comportamento in presenza di interruzioni (gaps) nella fila di piante e al variare dell'illuminazione.
- Casi limite riscontrati (es. testate del campo, biforcazioni del corridoio, erbacce invadenti).

8.3 Discussion

8.3.1 Segmentation performance

The architecture comparison supports LR-ASPP as an accuracy–speed compromise: on the manual lettuce test images it loses about 1.1 percentage points of mIoU with respect to FCN and DeepLabV3, with much shorter recorded inference times. IoU alone does not establish that a segmentation is sufficient for navigation. This is assessed through the geometric extraction and the robot tests.

The supervision comparison tested labels generated by SAM 3 and used without any manual correction. On lettuce, networks trained on these labels remain below the manual model (path IoU 0.9013 against 0.9587 for the binary network in the realistic scenario) and lose most of their accuracy on the complete test set. Part of this gap may come from a difference in label definition: the manual masks mark only the central traversable corridor as path, whereas networks trained on SAM 3 labels can also segment adjacent or non-traversable corridors, which the IoU counts as false positives. This behavior was observed visually for the deployed network but was not quantified. The deployed network required no manual annotation of its training data; the time saved with respect to manual labeling was not measured.

8.3.2 Geometric extraction trade-offs

The four geometric extraction methods involve different trade-offs, and none is best on every criterion:

- **Method 4 versus `pf_ransac`:** Method 4 is deterministic and needs no random hypothesis generation; `pf_ransac` is reproducible only through its fixed seed and evaluates all boundary points in each of its 100 trials. `pf_ransac` operates on row-wise boundary points without horizontal discretization or

sequential gating. When canopy boundaries contain extended irregular gaps, RANSAC can still find inliers above and below the gap, whereas Method 4 stops the association after more than three consecutive strips without a compatible candidate.

- **Method 4 versus `pf_centri`:** By maintaining separate linear models for the left and right corridor boundaries, Method 4 isolates asymmetric disturbances such as localized weed patches or protruding leaves. In contrast, `pf_centri` collapses interval endpoints into midpoints prior to fitting, causing unilateral border noise to directly perturb the estimated centerline. Nevertheless, `pf_centri` requires fewer validation checks and operates with a simpler association logic than the decoupled dual-edge architecture.
- **Method 4 versus `pf_histogram`:** The column-histogram baseline (`pf_histogram`) is computationally the simplest approach, computing column projections in $O(W \cdot H)$ without strip partitioning, boundary association, or matrix inversion. It does not suffer from tracking divergence or initialization failure. Its primary drawback is geometric incompleteness: it produces only a horizontal setpoint x_{target} without estimating heading error e_θ. Furthermore, its target shifts when vegetation area varies unevenly along the corridor. Method 4 delivers a complete linear model (m_C, q_C) supporting full pose feedback, but incurs higher algorithmic complexity and depends on bottom-up mask continuity.

8.3.3 Limitations

The experimental findings and the proposed methodology are subject to several specific limitations:

- **Seed dependency in Method 4:** The bottom-up tracking logic relies on detecting a valid corridor interval within the bottom six strips of the region of interest. If the mask immediately in front of the vehicle is corrupted by motion blur, ground glare, or chassis shadows, tracking fails to initialize, causing frame rejection even when the upper corridor is clearly segmented. Unlike global projection methods, Method 4 lacks a mechanism to re-anchor tracking from the top down.
- **Fixed spatial gate calibration:** The continuity threshold $\Delta x_{\max} = 28$ px and strip discretization parameters are fixed in image coordinates. Large vehicle pitch or roll on rough terrain causes rapid lateral shifts of the corridor boundaries between consecutive frames. When these shifts exceed the gate, the tracker rejects valid boundaries.

- **Straight-line corridor assumption:** All linear models evaluated (Method 4, `pf_ransac`, and `pf_centri`) assume locally straight, parallel corridor boundaries. They cannot follow curved planting geometries or guide the robot through headland turnaround maneuvers where corridor boundaries terminate or diverge.
- **Segmentation evaluation scope:** Quantitative segmentation evaluation is limited to the lettuce test images. The deployed weights have no independent IoU, because their training data include the lettuce test images; the multi-crop configuration is evaluated through a separate network trained without those images, and tomato and cucumber have no manual ground truth. The deployed network was also assessed by visual inspection of its predictions, in simulation, and in field tests. In addition, the random image-level split takes training and test images from the same lettuce folders, each recorded along one path; no test image is used for training, but the reported IoU values may overestimate performance on paths or fields not seen during training.
- **Offline versus closed-loop dynamics:** Offline benchmarks quantify reference line accuracy on static recorded frames. They do not account for dynamic wheel-ground interaction, actuator saturation, or latency jitter during physical vehicle operation.

Chapter 9

Conclusions and Future Work

This chapter summarizes the main findings, discusses the contributions and limitations of the work, and identifies directions for future research.

9.1 Main contributions

This thesis investigated vision-based local guidance for agricultural row navigation through segmentation model selection, dataset preparation with automatic SAM 3 annotation, and geometric reference extraction. The main contributions are:

- **Architecture comparison for robotic segmentation:** FCN, DeepLabV3, and LR-ASPP were compared on manually annotated lettuce images. LR-ASPP was selected for its accuracy–speed compromise: on the test images it reached a path IoU of 0.9532 with a recorded inference time of 8.21 ms, against 59.10 ms and 86.90 ms for the other architectures, at the cost of an mIoU about 1.1 percentage points lower.
- **Automatic dataset annotation with SAM 3:** Training labels were generated with SAM 3 text prompts and used without any manual correction. Networks trained on these labels for lettuce were compared with a network trained on manual masks, and the binary labeling was then extended to the complete lettuce, tomato, and cucumber collection to train the network deployed on the robot.
- **Evaluation of output label configurations:** Networks with four-class, three-class, and binary path outputs, trained on SAM 3 lettuce labels, were compared against the manual ground truth. Their path IoU values differ by less

than 0.5 percentage points in the realistic scenario. The binary configuration was adopted for the robot because the path is the only class used by the geometric extraction.

- **Edge-based geometric extractor:** An extractor that converts corridor masks into image-space navigation references was implemented (`pf5.py`, Method 4). It uses horizontal strip extraction, bottom-up interval association with selection of the chain that gives the most edge samples, and independent fitting of the left and right boundaries, and it is compared with three baseline algorithms (`pf_centri`, `pf_histogram`, `pf_ransac`).

9.2 Conclusions on segmentation and annotation

The architecture comparison supports LR-ASPP as a compromise between segmentation accuracy and recorded inference time on the robot hardware. Whether its segmentation is sufficient for navigation is assessed through the geometric extraction and the robot tests, not from IoU alone.

On lettuce, the manual model remains the best: path IoU 0.9532 on the complete test set and 0.9587 in the realistic scenario. Networks trained only on uncorrected SAM 3 labels reach a path IoU of about 0.90 in the realistic scenario (0.9013 for the binary network), but only 0.39–0.41 on the complete test set, and the 4-class network reaches an mIoU of 0.5925. Fully automatic labels therefore provide path supervision of about 0.90 IoU in the operational conditions, but they are not a replacement for manual annotation when all classes are needed or when the scene departs from those conditions. The multi-crop binary configuration used on the robot, trained on the complete SAM 3 collection without any manual annotation, reaches 0.9071 in the realistic scenario and 0.6369 on the complete test set when the lettuce test images are excluded from its training data. The deployed weights were assessed visually, in simulation, and on the robot.

The choice of output label configuration affects both annotation effort and downstream geometric extraction. Binary path segmentation focuses the model directly on the navigation target, simplifying both the labeling task and the geometric stage. This comes at the cost of discarding plant and covering-material information that might be useful for other tasks.

9.3 Conclusions on geometric extraction

The geometric extraction stage converts binary corridor segmentation masks into navigation setpoints. The comparative study examined four distinct architectural paradigms:

1. Global column projection (`pf_histogram`) maximizes simplicity and eliminates tracking failure modes, but discards heading angle information and exhibits sensitivity to asymmetric vegetation distribution.

2. Centerline strip regression (`pf_centri`) provides both lateral offset and heading estimates with low computational demand, but transfers unilateral canopy noise directly into the navigation line.

3. Random sample consensus (`pf_ransac`) handles scattered boundary points and wide gaps without strip discretization, but relies on random sampling: its result is reproducible only with a fixed seed, and each trial evaluates all boundary points.

4. Decoupled dual-edge tracking (Method 4) provides deterministic execution and boundary-level noise decoupling, but introduces strict dependencies on near-field seed detection and heuristic jump gates.

These characteristics demonstrate that no individual extractor is universally superior; selection depends on sensor setup, platform compute limits, and field layout.

▶ DA COMPLETARE: Sintesi quantitativa finale dell'estrazione geometrica e navigazione

Sintetizzare i dati conclusivi:

- Margine di precisione medio dell'asse stimato (scostamento in pixel e in centimetri).
- Tasso medio di affidabilità e di reiezione dei frame lungo i test eseguiti.
- Conferma della sostenibilità del controllo a 20 Hz su piattaforma embedded Jetson in condizioni di campo.

9.4 Limitations

The methodology and experimental findings are subject to several specific limitations:

- **Bottom-up seed dependency in Method 4:** The tracking algorithm initializes exclusively from the bottom six horizontal strips. Occlusions immediately in front of the camera (such as robot chassis shadows, soil buildup, or puddles) prevent track initialization and lead to frame rejections, even when the distant corridor is visible.

- **Geometric model constraints:** Method 4, `pf_ransac`, and `pf_centri` assume locally straight, parallel corridor boundaries. They cannot follow

curved crop rows or navigate headland turnaround maneuvers where crop boundaries terminate or diverge.

- **One-sided gap vulnerability:** Method 4 tracks whole intervals, so both edges are associated together. If missing plants on one side make the edge displacement exceed the limit in more than three consecutive strips (18 px), association stops, and the frame is rejected if either edge has fewer than two non-clipped samples. There is no single-edge fallback, whereas the column histogram still provides a lateral setpoint.
- **Image-space gate tuning:** The continuity gate ($\Delta x_{\max} = 28$ px) is defined in image coordinates. Large vehicle pitch and roll on uneven soil cause sudden lateral displacements of the corridor between consecutive scanlines, which can trigger premature tracking loss.
- **Dataset annotation coverage:** Segmentation accuracy was measured against manual ground truth only on lettuce. The deployed weights have no independent IoU, because their training data include the lettuce test images; the multi-crop configuration was evaluated through a separate network trained without them. Tomato and cucumber have no manual ground truth. The random image-level split also takes training and test images from the same lettuce folders, each recorded along one path, so the reported IoU values may be optimistic for paths not seen during training.
- **Label definition mismatch:** The manual masks mark only the central traversable corridor as path, whereas networks trained on SAM 3 labels can also segment adjacent or non-traversable corridors. This lowers their IoU against the manual masks, but the size of the effect was not measured.
- **Quality of the SAM 3 labels:** Visual inspection showed that SAM 3 segmented the plastic mulch film poorly and also assigned the greenhouse plastic cover to the covering class, and that its path masks were less accurate in images taken from unusual viewpoints or where the path between the rows is not clearly visible. Because the labels were used without correction, these errors enter the training data; their extent was not measured.
- **Annotation effort measurement:** The time saved by automatic labeling with respect to manual annotation was not measured.
- **Closed-loop dynamics:** Offline benchmarks evaluate reference accuracy on static test images. They do not account for vehicle inertia, steering backlash, or dynamic latency variations under closed-loop field control.

9.5 Future work

Several directions for future research emerge from this work:

9.5.1 Dataset and annotation

- Measure the annotation time of manual and automatic workflows, and test whether a light correction of the SAM 3 labels improves the trained networks.
- Split the data by folder rather than by image, so that the test paths are not represented in training, and repeat the evaluation of all networks.
- Prepare manual ground-truth masks for tomato and cucumber to evaluate the deployed network on those crops directly.
- Investigate active learning strategies to select images for manual annotation that improve model generalization most effectively.
- Explore other foundation models or fine-tuned SAM variants specialized for agricultural segmentation.

9.5.2 Segmentation and model deployment

- Evaluate additional lightweight architectures (e.g., BiSeNet, FastSCNN) for further latency reduction.
- Investigate quantization and model compression techniques to enable deployment on edge devices.
- Test temporal consistency methods that use previous frames to improve segmentation and reduce flickering.
- Explore uncertainty estimation to detect when segmentation confidence is low and fallback strategies are needed.

9.5.3 Geometric extraction

- Extend the geometric representation from linear models to piecewise linear or polynomial formulations to support curved rows and headland turns.
- Implement top-down recovery logic and single-edge asymmetric tracking to maintain guidance when bottom-up seeding fails or unilateral row dropouts occur.

- Adapt spatial gate thresholds dynamically using vehicle attitude telemetry from an onboard IMU to prevent false tracking loss during pitch and roll variations.
- Incorporate temporal Bayesian tracking (such as an Extended Kalman Filter) to smooth reference parameters across frame dropouts.

9.5.4 System integration and field validation

- Complete closed-loop field trials to validate end-to-end navigation performance under variable lighting, growth stages, and row conditions.
- Measure lateral tracking error and success rate over extended distances and multiple crop types.
- Integrate temporal filtering and prediction to handle brief reference loss or occlusions.
- Investigate fusion with other sensors (e.g., depth, IMU) to keep a valid reference when the camera view is degraded.
- Develop a complete autonomy system including global path planning, obstacle avoidance, and task-specific behaviors.

9.6 Closing remarks

This thesis addressed the segmentation, dataset preparation, and geometric extraction stages of vision-based agricultural navigation. The selection of LR-ASPP balances accuracy and speed for robotic deployment. Labels generated by SAM 3 without manual correction were used to train the network deployed on the robot, and the geometric extractor converts masks into navigation references using horizontal strip selection and independent edge fitting.

The work establishes a foundation for camera-based local guidance in structured crop environments. Future research should complete field validation, extend the geometric model to handle curves and gaps, and evaluate the deployed network on independent, sequence-level test data.

Bibliography

[1] Yuhao Bai, Baohua Zhang, Naimin Xu, Jun Zhou, Jiayou Shi, and Zhihua Diao. «Vision-based navigation and guidance for agricultural autonomous vehicles and robots: A review». In: *Computers and Electronics in Agriculture* 205, 107584 (Feb. 2023). DOI: 10.1016/j.compag.2022.107584 (cit. on pp. 1, 2, 4, 5, 9, 13, 53).

[2] Rajitha de Silva, Grzegorz Cielniak, Gang Wang, and Junfeng Gao. «Deep Learning-Based Crop Row Detection for Infield Navigation of Agri-Robots». In: *Journal of Field Robotics* 41.7 (2024), pp. 2299–2321. DOI: 10.1002/rob.22238 (cit. on pp. 1, 4, 7–10, 13, 15, 16).

[3] Alessandro Navone, Mauro Martini, Marco Ambrosio, Andrea Ostuni, Simone Angarano, and Marcello Chiaberge. «GPS-Free Autonomous Navigation in Cluttered Tree Rows with Deep Semantic Segmentation». In: *Robotics and Autonomous Systems* 183, 104854 (Jan. 2025). DOI: 10.1016/j.robot.2024.104854 (cit. on pp. 1, 2, 4, 7, 8, 10–12, 15, 16, 53).

[4] Enrico Mendez, Javier Piña Camacho, Jesús Arturo Escobedo Cabello, and Alfonso Gómez-Espinosa. «Autonomous Navigation and Crop Row Detection in Vineyards Using Machine Vision with 2D Camera». In: *Automation* 4.4 (Sept. 2023), pp. 309–326. DOI: 10.3390/automation4040018 (cit. on pp. 2, 5, 7–9, 12, 15, 16).

[5] Johannes Kneip, Patrick Fleischmann, and Karsten Berns. «Crop Edge Detection Based on Stereo Vision». In: *Robotics and Autonomous Systems* 123, 103323 (Jan. 2020). DOI: 10.1016/j.robot.2019.103323 (cit. on pp. 4, 7, 12, 14–16).

[6] Xia Li, Junhao Su, Zhenchao Yue, and Fangtao Duan. «Adaptive Multi-ROI Agricultural Robot Navigation Line Extraction Based on Image Semantic Segmentation». In: *Sensors* 22.20, 7707 (Oct. 2022). DOI: 10.3390/s22207707 (cit. on pp. 5, 7, 10, 12, 14, 15, 17, 54).

[7] Jiqing Chen, Hu Qiang, Jiahua Wu, Guanwen Xu, and Zhikui Wang. «Navigation Path Extraction for Greenhouse Cucumber-Picking Robots Using the Prediction-Point Hough Transform». In: *Computers and Electronics in Agriculture* 180, 105911 (Jan. 2021). DOI: 10.1016/j.compag.2020.105911 (cit. on pp. 5, 7, 12, 15, 16).

[8] Alessandro Navone, Mauro Martini, Andrea Ostuni, Simone Angarano, and Marcello Chiaberge. «Autonomous Navigation in Rows of Trees and High Crops with Deep Semantic Segmentation». In: *2023 European Conference on Mobile Robots (ECMR)*. IEEE, 2023, pp. 1–6. DOI: 10.1109/ECMR59166.2023.10256334 (cit. on p. 10).

[9] Jonathan Long, Evan Shelhamer, and Trevor Darrell. «Fully Convolutional Networks for Semantic Segmentation». In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2015, pp. 3431–3440. DOI: 10.1109/CVPR.2015.7298965 (cit. on pp. 10, 18–22).

[10] Liang-Chieh Chen, George Papandreou, Florian Schroff, and Hartwig Adam. «Rethinking Atrous Convolution for Semantic Image Segmentation». In: *arXiv preprint arXiv:1706.05587* (2017) (cit. on pp. 10, 20, 22–24).

[11] Andrew Howard et al. «Searching for MobileNetV3». In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2019, pp. 1314–1324 (cit. on pp. 10, 24–27, 29).

[12] Alexander Kirillov et al. «Segment Anything». In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. Oct. 2023, pp. 4015–4026. arXiv: 2304.02643 [cs.CV] (cit. on pp. 11, 30, 31).

[13] Nicolas Carion et al. «SAM 3: Segment Anything with Concepts». In: *International Conference on Learning Representations (ICLR)*. 2026. arXiv: 2511.16719 [cs.CV]. URL: https://proceedings.iclr.cc/paper_files/paper/2026/hash/e0982cbc81401df3430ee1ff780dc7a2-Abstract-Conference.html (cit. on pp. 11, 30, 31).

[14] Letian Wu, Yongzhi Cui, Huifeng Shi, Xiaoli Sun, Jiayan Yang, Xinwei Cao, Ping Zou, and Ya Liu. «An Improved DeepLabV3+-Based Method for Crop Row Segmentation and Navigation Line Extraction in Agricultural Fields». In: *Sensors* 26.10, 3142 (May 2026). DOI: 10.3390/s26103142 (cit. on pp. 13, 53).

[15] Martin A. Fischler and Robert C. Bolles. «Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography». In: *Communications of the ACM* 24.6 (June 1981), pp. 381–395. DOI: 10.1145/358669.358692 (cit. on pp. 13, 53).

[16] Aurélien Géron. *Hands-On Machine Learning with Scikit-Learn and PyTorch: Concepts, Tools, and Techniques to Build Intelligent Systems*. 1st ed. O'Reilly Media, 2026. ISBN: 979-8-341-60798-9 (cit. on pp. 18–21, 23, 39, 40).

[17] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. «Deep Residual Learning for Image Recognition». In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 770–778. DOI: 10.1109/CVPR.2016.90 (cit. on p. 20).

[18] TorchVision Maintainers and Contributors. *TorchVision: Models and Pretrained Weights — FCN-ResNet50*. URL: https://pytorch.org/vision/stable/models/fcn.html (visited on 10/23/2026) (cit. on pp. 22, 38).

[19] TorchVision Maintainers and Contributors. *TorchVision: DeepLabV3 segmentation models, source code* ***torchvision/models/segmentation/deeplabu3.py***. URL: https://github.com/pytorch/vision/blob/main/torchvision/models/segmentation/deeplabv3.py (visited on 10/23/2026) (cit. on pp. 24, 38).

[20] TorchVision Maintainers and Contributors. *TorchVision: LR-ASPP segmentation model, source code* ***torchvision/models/segmentation/lraspp.py***. URL: https://github.com/pytorch/vision/blob/main/torchvision/models/segmentation/lraspp.py (visited on 10/23/2026) (cit. on pp. 27, 38).

[21] Devansh Arpit et al. «A Closer Look at Memorization in Deep Networks». In: *Proceedings of the 34th International Conference on Machine Learning*. Ed. by Doina Precup and Yee Whye Teh. Vol. 70. Proceedings of Machine Learning Research. 2017, pp. 233–242. URL: https://proceedings.mlr.press/v70/arpit17a.html (cit. on p. 42).

[22] NumPy Developers. *numpy.polyfit: NumPy v2.4 Manual*. URL: https://numpy.org/doc/2.4/reference/generated/numpy.polyfit.html (visited on 09/13/2026) (cit. on p. 49).